

Linguistics Institute
Computational Linguistics Department
Heinrich-Heine-University Düsseldorf
Universitätsstr. 1, 40225 Düsseldorf (Germany)



From Benchmark to Browser:
A Comparative Evaluation of
Automatic Text Simplification
Approaches for *DOM-Preserving*
Web Deployment

Yunus Oscar Renz
Student ID: 2952305
Email: yuren100@hhu.de

Bachelor's Thesis
Computational Linguistics

Date of submission: September 30th, 2026
Examiners: apl. Prof. Dr. Wiebke Petersen
Dr. Kilian Evang

Abstract

Automatic text simplification (ATS) rewrites a text to make it easier to read while preserving its meaning. ATS research, however, has been dominated by sentence-aligned Wikipedia corpora, which differ from the web pages on which readers encounter complex text. Web pages add a layer of complexity that no benchmark contains: continuous prose interleaved with headings, navigation and form controls. This thesis therefore asks what changes when simplification moves from benchmarks to a user-facing browser extension.

Three approaches are compared at sentence and document level: an existing simplifier, a sequence-to-sequence model fine-tuned in this thesis, and prompted open-weight large language models (LLMs). They are evaluated with automatic metrics on public benchmarks and by structured manual inspection of the outputs. All three are served through a browser extension backed by a local inference service, which simplifies web pages in place.

After fine-tuning, the 139M-parameter BART-base scores significantly higher than its untuned checkpoint on the sentence-level simplification metric SARI and on document-level SARI (D-SARI). It scores similarly to the existing simplifier, three times its size, and below both prompted LLMs (3B and 7B parameters) on SARI. At the document level, the 7B model scores higher than the fine-tuned model on LENS, a metric trained on sentence-level human judgements and applied here out of domain. On D-SARI, by contrast, it scores below the fine-tuned model, so no single definitive ranking emerges: at document level, the ordering depends on the metric. The inspection also found a factual error that no metric reflects: a checkpoint trained on less data changed the census year 2010 to 2000 in 27 of 35 affected documents.

To test real-world behaviour, an exploratory site audit runs the extension on twelve authentic web pages at both levels. The page structure was preserved in all 24 runs: no tag, image source or link target was lost, and reverting restored the page’s text and attributes. The audit nevertheless exposed failures invisible to benchmark scores: content selection silently withheld ordinary prose that contains a semicolon, and five sentences were served with a negation deleted, reversing their meaning without triggering any guard.

Three observations plausibly transfer beyond these models: training and deployment should agree on the unit of text, benchmark and metric choice can reverse a conclusion, and reading the outputs finds failures that scores miss, such as the census-year substitution and the negation deletions.

Acknowledgments

First and foremost, I would like to express my sincere gratitude to apl. Prof. Dr. Wiebke Petersen (Heinrich Heine University Düsseldorf), my first examiner and one of my biggest supporters throughout my academic journey. Thank you for the support you provided, for pushing me when I needed to be pushed, and always believed in me. Your guidance, encouragement, and support really made a difference.

Additionally, I would like to thank my second examiner, Dr. Kilian Evang (Heinrich Heine University Düsseldorf), for his support, advice, and willingness to be there whenever I needed guidance.

My sincere thanks also go to Dr. Regina Stodden (Bielefeld University) for her guidance, valuable input, and the impulses she provided for this project. I am especially grateful that you took the time to support me shortly before your parental leave.

Beyond the academic support I received, I would like to thank my family, my friends, and my coworkers for always having my back and supporting me throughout this period of my life. Your encouragement and understanding made this journey considerably easier.

Finally, I want to thank all of my study buddies who spent countless hours with me in cafés, libraries, and wherever else we could find a place to work. The discussions, shared struggles, coffee breaks, and hours spent working alongside each other made this project not only possible, but also a lot more enjoyable.

To everyone who supported me along the way: thank you.

Contents

List of Figures	vi
List of Tables	vii
List of Abbreviations	viii
List of Symbols	x
1 Introduction	1
1.1 Motivation: The non-simple web	1
1.2 Research questions and scope	2
1.3 Contributions	3
1.4 Thesis structure	4
2 Background	5
2.1 Automatic text simplification	5
2.2 Neural simplification: trained and instructed models	6
2.3 Parallel corpora for text simplification	7
2.4 Evaluating text simplification	7
2.5 Web pages as input	9
3 Related Work	10
3.1 Approaches to text simplification	10
3.2 Browser-based deployment	12
3.3 Behavioural robustness beyond held-out accuracy	12
3.4 Research hypotheses and expectations	13
4 Methodology I: Simplification Approaches	14
4.1 Comparison design	14
4.2 Corpus assessment and preparation	14
4.3 Fine-tuning sequence-to-sequence models	17
4.4 Prompted open-weight language models	19
4.5 Evaluation protocol	21
5 Methodology II: Browser Extension Deployment	23
5.1 Architecture and local-processing constraints	23
5.2 Content selection and DOM-preserving replacement	25
5.3 Unit of simplification	26
5.4 Output guards and execution control	29
5.5 Instruments for evaluating deployment	29

6	Results	31
6.1	Corpus assessment	31
6.2	Sentence-level results	31
6.3	Document-level results	34
6.4	Qualitative analysis of document-level outputs	36
6.5	Qualitative analysis of sentence-level outputs	39
6.6	Deployment results: coverage, preservation, and selection . . .	40
6.7	Granularity and input-unit findings	44
6.8	Runtime and cost	45
7	Discussion	48
7.1	RQ1: Comparing the three approaches	48
7.2	RQ2: Effective strategies and behavioural robustness	51
7.3	Cross-cutting findings	52
8	Limitations and Threats to Validity	54
9	Future Work	57
10	Conclusion	60
	References	62
	Appendices	68
	Statement of Originality	78
	Declaration of AI Usage	78

List of Figures

4.1	Source-Document Length Distribution Relative to the 512-Token Input Cap	18
5.1	System Architecture and the Local-Processing Constraint . .	24
5.2	The Request Path, and Where the Granularities Diverge . . .	24
6.1	Relationship Between BLEU, SARI, and the Unchanged Rate	33
6.2	Sensitivity of BLEU to the Number of References	33
6.3	System Rankings Under D-SARI and LENS	35
6.4	Validation Loss of the Full-Scope Document Run as a Proxy for D-SARI	36
6.5	Dependence of Output Compression on Source Length	37
6.6	Per-Page Variation in Skip Share Across the Two Cuts	41
6.7	Composition of Hand-Labelled Verdicts for Skipped and Unchanged Items	42
6.8	Latency Profiles of Four Model Configurations	47

List of Tables

4.1	Corpus Assessment and Selection Decisions	15
4.2	Comparison of WikiLarge Split Sizes Under the Two Mojibake Filters	16
4.3	Training Configuration by Granularity, Scope, and Device . .	18
5.1	Segment-Join Strategies and Their Failure Modes	27
5.2	Effect of Input Convention on Document-Model Output . . .	28
6.1	Sentence-Level Comparison of the Five Conditions on ASSET	32
6.2	Document-Level Comparison on D-Wikipedia at Two Training Scopes	34
6.3	Surface Probes of Document Outputs Across Checkpoints and References	37
6.4	Attribution of Document-Cut Symptoms to Pipeline Layers .	45
7.1	Score Differences Between the Fine-Tuned Checkpoint and the Other Systems	48
A.1	Decoding Configurations for Serving and for Evaluation . . .	70
A.2	In-Context Demonstrations and the Operations They Cover .	71
A.3	Hand-Authored Demonstrations for the Four Non-Default Audiences	72
A.4	The Twelve Audited Pages, by Category and URL	73
A.5	Per-Page Coverage Counts in Both Cuts	74
A.6	Hand-Labeling Codebook: Verdict Criteria and Counts . . .	75
A.7	Unguarded Meaning Inversions in the Sentence Cut	76
A.8	Recorded Outputs of the Document Model Under Five Input Conventions	77

List of Abbreviations

- ACCESS** AudienCe-Centric Sentence Simplification 6, 10, 12, 58
- API** application programming interface 19, 26, 43, 73
- ATS** automatic text simplification ii, 1–3, 5, 23, 48, 60, 61
- BART** Bidirectional and Auto-Regressive Transformers ii, 2, 6, 10–14, 17, 19, 54, 59, 60
- BLESS** Benchmarking Large language models on Sentence Simplification 6, 8, 10, 11, 13, 19–21, 68, 70, 71
- BLEU** Bilingual Evaluation Understudy 7, 13, 32, 34, 35, 50, 53, 55
- CI** confidence interval 31, 48, 60
- CPU** central processing unit 45, 56
- CSS** Cascading Style Sheets 73
- D-SARI** document-level SARI ii, 8, 11, 13, 21, 22, 34–36, 38, 48–50, 53–55, 57, 60
- DOM** Document Object Model 2, 27, 30, 51, 53, 60, 61
- FKGL** Flesch–Kincaid Grade Level 5, 8, 21, 22, 29, 31, 32, 34, 55
- gibibyte (GiB)** binary unit of memory, 2^{30} bytes (about 7% more than a gigabyte), as reported by the training framework 17
- gigabyte (GB)** decimal unit of memory, 10^9 bytes, used for file and machine sizes 19
- GPU** graphics processing unit 22, 45, 57
- HF** Hugging Face 2, 16, 19
- HTML** HyperText Markup Language 26, 30, 40, 51, 73
- LENS** Learnable Evaluation metric for Text Simplification ii, 8, 13, 21, 35, 48–50, 53–57, 60
- LLM** large language model ii, 3, 6, 11–14, 19, 20, 24, 31, 40, 48–50, 59–61
- MPS** Metal Performance Shaders 18

MUSS Multilingual Unsupervised Sentence Simplification 10, 13

PWKP Parallel Wikipedia Simplification 7, 14, 15, 20, 39

SARI System output Against References and against the Input sentence ii, 8, 10, 13, 19, 21, 22, 31–34, 48–50, 55, 60

seq2seq sequence-to-sequence ii, 3, 6, 10, 11, 13–15, 19, 29, 40, 45, 48, 59–61

T5 Text-to-Text Transfer Transformer 11, 54

TAACO Tool for the Automatic Analysis of Cohesion 58

TF*IDF term frequency–inverse document frequency 15

URL uniform resource locator 25, 73

List of Symbols

- n -gram** Contiguous sequence of n items (such as words, characters, or tokens) from a sample of text or speech 7, 8, 13, 32, 48, 49, 54, 56
- F_{add} SARI’s F-score over the n -grams a system adds 8
- F_{keep} SARI’s F-score over the n -grams a system keeps 8
- P_{del} SARI’s precision over the n -grams a system deletes 8
- Δ SARI** Change in SARI against the untuned checkpoint, reported with a paired-bootstrap interval 32
- n Number of items in a sample or an evaluation split 28, 32, 34–37, 48, 55, 75, 77
- k Number of references a reference-overlap metric is computed against 32, 33
- p Significance level from the paired bootstrap, never the sampling threshold 13, 21, 31, 34, 48, 60
- top- p** Nucleus-sampling probability threshold, never the significance level 11, 70
- P_{90} 90th percentile of a distribution 15, 17
- P_{95} 95th percentile of a distribution, used for token lengths and request latency 15, 44, 46, 47, 52
- k** Thousand, as a suffix to a count (300k examples) 12, 15, 16, 18
- M** Million, as a suffix to a parameter count (139M parameters) ii, 10, 11, 32, 48, 49, 60
- B** Billion, as a suffix to a parameter count (7B parameters) ii, 19, 32, 34, 35, 45, 46, 48, 50, 53, 57, 59, 60
- fp16** 16-bit floating-point number format 18
- bf16** bfloat16, a 16-bit floating-point number format with the exponent range of 32-bit floats 18

1 Introduction

1.1 Motivation: The non-simple web

Web content is often written at a level of linguistic complexity that exceeds what some of its intended readers can readily process. West (2003, pp. 4–5), for example, found that the front page of the average US government website read at an 11th-grade level and that only 12 % of sites were at or below the 8th-grade level, the reading limit of half of the US public in 2003. Counterintuitively, sites potentially serving less educated audiences are not written in simpler language: 83 % of corrections sites, 74 % of elementary-education sites, and 69 % of housing and of health sites read at a 12th-grade level (West, 2003, p. 5).

Where prose is pitched above the reading level of the intended audience, the obstacle may lie not in the subject but in its linguistic packaging. Rink (2018, pp. 29, 31) describes such obstacles as *communication barriers*, among them a *cognitive barrier*, which arises when a message’s conceptual structure overtaxes its recipients cognitively. An important factor in this barrier is the linguistic or content-related complexity of the message (Schubert, 2016, pp. 18–19). Affected readers include people with cognitive or learning disabilities, aphasia or dementia, and people with limited functional literacy (Rink, 2018, p. 38), as well as readers with dyslexia or autism, but also non-native speakers, and children (Alva-Manchego, Scarton, & Specia, 2020, pp. 135, 137). Access to communication is a condition for participation in society (Rink, 2018, p. 29). Manual text simplification is resource-intensive (Stodden, 2026, p. 21), and for continuously changing web content doesn’t scale.

This creates a role for automatic text simplification (ATS), which rewrites text to make it easier to understand while keeping its essential meaning (Siddharthan, 2014, p. 259; Alva-Manchego, Scarton, & Specia, 2020, p. 135). Meaning preservation is therefore a central constraint of the task: a rewrite that is easier to understand but changes what the source communicates would not fulfil the intended function of text simplification. Xu et al. (2016, p. 402) note that Simple English Wikipedia data has dominated simplification research since 2010.

Web-page simplification requires a deployed simplifier to determine which textual units to simplify and to write the resulting output back without disrupting the page’s Document Object Model (DOM) structure (Section 2.5). Real web pages contain heterogeneous content such as headings, lists, and captions (Hellbusch, 2018, pp. 518–519). A simplifier trained exclusively on aligned sentence pairs may therefore not have learned to handle such non-sentence page elements. The challenge is consequently not only to produce simpler text, but also to ensure that the output remains structurally and semantically aligned with the input and can be integrated into the page without breaking its structure or functionality.

These problems became apparent during different stages of this thesis. For example, in early implementation experiments, an existing Bidirectional and Auto-Regressive Transformers (BART)-based simplifier from the Hugging Face (HF) Hub produced “Other websites” for isolated fragments such as “By”, “on”, and “Published”. Later, the sentence-level model fine-tuned in this thesis turned the heading “Municipal recycling policy revisions” into the same phrase. These outputs were not caused by an error in the browser extension or the underlying infrastructure, but could be traced to the supervision data both models were trained on, as discussed in Section 6.5. Later, when the fine-tuned models were tested on authentic web pages, the site audit identified sentences that were served unchanged except for a deleted negation. The deletions reversed their meaning (Section 6.6).

Such failures illustrate a limitation of evaluating ATS only through benchmark scores: a flawed but well-formed output can pass through an automated evaluation without revealing that it failed. Held-out accuracy often overestimates how well a model performs outside its evaluation distribution (Ribeiro et al., 2020, p. 4902).

1.2 Research questions and scope

This thesis evaluates ATS at the benchmark and at the deployment level, drawing on three kinds of evaluation: automatic metrics on public benchmarks, structured manual inspection, and an audit of the browser extension on authentic web pages. Two research questions organise the work.

RQ1 (Comparative performance): How do different ATS methods compare with each other at sentence and document granularity, evaluated using established automatic metrics and structured manual inspection?

RQ2 (Deployment behaviour): Which content-selection and DOM-preserving replacement strategies enable reliable application of ATS to real-world web pages, and what does an audit of the deployed system on such pages show about its robustness in practice?

Both questions are answered on English text, over a grid of three approaches against the two granularities (Section 4.1). The approaches are an off-the-shelf sequence-to-sequence (seq2seq) model for simplification (the *comparison simplifier*); the author’s seq2seq model fine-tuned on a parallel dataset; and prompted open-weight large language models (LLMs). Unlike the two seq2seq models, the prompted LLMs are general-purpose instruction models that were never trained for simplification specifically and receive the task through prompting with an instruction and a few demonstrations. I derive four hypotheses (Section 3.4): fine-tuning should improve simplification-specific metrics over the same checkpoint before fine-tuning (H_1), a prompted LLM should match or exceed a fine-tuned model (H_2), reference-overlap and simplification-specific metrics should rank differently (H_3), and behaviour on authentic pages should diverge from benchmark behaviour (H_4).

1.3 Contributions

In this thesis I make these contributions to the evaluation and deployment of ATS. This thesis ...

... provides a **systematic assessment of the corpora used in the experimental setup**, documenting data-quality issues that affect their suitability for ATS (Sections 4.2 and 6.1).

... provides a **controlled comparison of three ATS approaches at sentence and document granularity**. The approaches are evaluated through a common evaluation harness, including the complete 8,000-document document-level test split. The reimplemented document-level metric is tested against its reference implementation (Sections 4.3, 4.4, 4.5 and 6.3).

... provides a **DOM-preserving browser deployment for comparing simplification approaches on authentic web pages**. This deployment makes it possible to examine content selection, simplification granularity, and replacement behaviour in the context in which such a system would actually be used, complementing evaluation on curated benchmarks (Sections 5.3 and 6.6).

... derives **methodological findings about the evaluation of ATS**. In particular, the results show how conclusions can depend on benchmark and granularity choices, and how structured inspection of system outputs can reveal behaviour that is not captured by automatic metrics alone (Section 7.3).

1.4 Thesis structure

Following the motivation, research questions, and core contributions outlined in Chapter 1, the remainder of this thesis is structured as follows:

Chapter 2 establishes the necessary background by introducing automatic text simplification, trained versus instructed neural models, the parallel corpora utilized, evaluation metrics, and the unique challenges of using web pages as input. Chapter 3 then reviews related work across simplification approaches, browser-based deployments, and behavioural robustness, culminating in the formalization of the core research hypotheses.

The core technical and empirical contributions are divided across two methodology chapters: Chapter 4 details the experimental design, corpus preparation, fine-tuning of sequence-to-sequence architectures, and the prompting strategy for open-weight language models, alongside the evaluation protocol. Subsequently, Chapter 5 presents the architecture and deployment of the browser extension under local-processing constraints, detailing DOM-preserving content replacement, input-unit definitions, and output guards.

Chapter 6 reports the comprehensive empirical findings, ranging from automatic sentence- and document-level metrics to qualitative reviews and runtime performance. These results are critically analysed and mapped directly back to the core research questions in Chapter 7.

Finally, Chapter 8 addresses the limitations and threats to validity of this work, Chapter 9 outlines promising avenues for future research, and Chapter 10 concludes the thesis.

In addition, the appendices provide the complete evaluation harness, the full set of structured inspection results, and the comprehensive audit of the deployed system on authentic web pages.

2 Background

2.1 Automatic text simplification

ATS takes in a text and outputs a version that is easier to read, while preserving the main idea (Siddharthan, 2014, p. 259). The rewriting proceeds through transformations such as replacement, reordering, and splitting (Alva-Manchego, Scarton, & Specia, 2020, pp. 135, 137–138):

- *Lexical operations* replace a rare or difficult word with a common or easier synonym.
- *Syntactic operations* include splitting longer sentences, reordering clauses, and converting passive to active voice.
- *Content-level operations* consist of deleting material judged to be peripheral or redundant.

The list is not exhaustive.¹ The last of these operations makes the task particularly challenging to evaluate, as deletion lowers the Flesch–Kincaid Grade Level (FKGL) (a readability formula, Section 2.4), but this does not necessarily make the output a better simplification (Alva-Manchego, Scarton, & Specia, 2020, pp. 175, 177).

The task is formulated at two granularities. In **sentence-level** simplification the training and evaluation units are single aligned sentence pairs, a narrower scope that has made corpora easier to collect and curate (Alva-Manchego, Scarton, & Specia, 2020, p. 136). In this field, most progress has been shown at this level (Vásquez-Rodríguez et al., 2024, p. 1). A system that simplifies one sentence at a time, however, cannot preserve cross-sentence cohesion. It cannot see those referential chains and discourse connectives across sentences, which lie outside its window. Alva-Manchego, Scarton, and Specia (2020, p. 136) note that document-level simplification arguably cannot be achieved by simplifying sentences one at a time.

In **document-level** simplification, the unit is an article paired with its simplified version. In the D-Wikipedia corpus an article is the lead section below the title, and pairs longer than 1,000 words are excluded (Sun et al., 2021, p. 7999).

¹Operations can also apply at the morphological level (Cardon & Bibal, 2023, p. 117); for German, long, one-token compounds are frequent, and their segmentation has been identified as a language-specific operation (Stodden, 2026, p. 189).

A document-level model therefore sees the referential chains and discourse connectives that a sentence-level system loses, although access to this context does not guarantee that the model exploits it. The model is moreover permitted, indeed expected, to delete, merge, reorder, and split across sentence boundaries, so its output may have no positional correspondence to its source. The trade-off is between an evaluation unit that is tractable and a modelling unit more faithful to how text is actually read. Which of the two units a deployed system should feed its model is a separate question, taken up for web pages in Section 2.5.

2.2 Neural simplification: trained and instructed models

Both families of system compared here are transformer-based, built on the attention-only architecture of Vaswani et al. (2023, p. 1). They differ in whether the simplification behaviour is acquired by gradient update or elicited by instruction.

The first family is the pretrained encoder–decoder, fine-tuned on parallel data. BART (Lewis et al., 2020, p. 7871) is a denoising seq2seq model trained by corrupting text with an arbitrary noising function and learning to reconstruct the original, an objective that aligns closely with rewriting. Its released checkpoints carry learned absolute positional embeddings sized for 1,024 tokens (`max_position_embeddings` in the checkpoint configuration). The 1,024-token size becomes a binding positional limit at document scale.

The second family is the instruction-tuned decoder-only causal LLM, steered by a prompt that carries a task instruction and some demonstrations. In-context learning (Brown et al., 2020, pp. 1, 3) induces a task from demonstrations placed in the prompt without updating the model’s weights. The prompted approach used here follows Benchmarking Large language models on Sentence Simplification (BLESS) (Kew et al., 2023), the benchmark of prompted simplification described in Section 3.1. BLESS reports its best prompted models performing comparably with trained simplification baselines at sentence level (Kew et al., 2023, p. 13291). Whether this holds against a model fine-tuned on task-matched data at the document level remains open; Hypothesis H_2 formalizes this comparison.

Between the two families sits controllable simplification. Control tokens as introduced by AudienCe-Centric Sentence Simplification (ACCESS) (Martin et al., 2020, p. 4691) train control into the model by conditioning generation on attributes computed from each source–target pair, such as compression ratio, paraphrase amount, and lexical or syntactic complexity. The LLM approach instead asks for control by prompting its target audience. The distinction is *control trained in* versus *control asked for* (Sections 3.1 and 4.4).

2.3 Parallel corpora for text simplification

Five corpora were assessed for this project. WikiLarge (X. Zhang & Lapata, 2017) was adopted for sentence-level training, D-Wikipedia (Sun et al., 2021) for document-level training and evaluation, and ASSET (Alva-Manchego, Martin, et al., 2020) for sentence-level evaluation. WikiSmall (Zhu et al., 2010) and SWiPE (Laban et al., 2023) were rejected. Section 4.2 gives the specification of each and the reasoning behind the two rejections.

Three properties of these corpora constrain the results. First, ASSET supplies ten human references per sentence (Alva-Manchego, Martin, et al., 2020, p. 4671) while D-Wikipedia supplies one, so an n -gram metric computed against the two is not measuring the same quantity. Second, D-Wikipedia’s release is fully lowercased, Penn-Treebank-style pre-tokenised, and stored one document per line with no newline inside a document body. That convention differs from WikiLarge’s natural-case sentences and, more consequentially, from the web text the extension processes. Third, both Wikipedia-derived corpora are automatically aligned, and automatic alignment is a known corpus-quality risk (X. Zhang & Lapata, 2017, p. 589). Xu et al. (2015, p. 285) examined 200 sampled pairs of the Parallel Wikipedia Simplification (PWKP) corpus by hand and found half of them not to be simplifications at all: 17 % carried different meanings on the two sides, and a further 33 % were no simpler on the simple side. Section 6.4 reports an instance of the same class in D-Wikipedia’s own test split.

Nor are the adopted and rejected corpora independent of each other. WikiLarge was built by aggregating earlier collections, among them Zhu’s PWKP, the corpus distributed as WikiSmall (X. Zhang & Lapata, 2017, p. 588). Material from the corpus rejected here therefore survives inside the corpus adopted. Whether fine-tuning on corpora of this quality still clearly improves on the same checkpoint before fine-tuning is the question addressed by Hypothesis H_1 .

2.4 Evaluating text simplification

Reference-overlap metrics score an output by its n -gram overlap with human references. Bilingual Evaluation Understudy (BLEU) (Papineni et al., 2002, p. 312) measures modified n -gram precision against one or more references. It is computed here through sacreBLEU (Post, 2018, p. 186), which applies a fixed tokenisation and preprocessing, so the scores are comparable with other sacreBLEU scores. BLEU does not compare the output with the input, so it favours conservative, low-edit output (Xu et al., 2016, p. 410). It is therefore a poor primary metric for a task where change is the goal. It is reported here as a documented counterexample and not as evidence of quality.

System output Against References and against the Input sentence (SARI) (Xu et al., 2016, pp. 404–405) was designed for simplification specifically. It scores the n -grams a system *adds*, *keeps*, and *deletes* against both the source and the references. The three scores are combined as the arithmetic mean of F_{add} , F_{keep} , and P_{del} , so a system that copies its input scores poorly rather than well. Deletion enters as a precision term rather than an F-score. SARI is the primary sentence-level metric here. Because the two metrics treat copying so differently, they can rank the same outputs in different orders, a phenomenon captured by Hypothesis H_3 .

Readability formulas measure the surface only. The FKGL estimates a school grade level from the average sentence length and the average number of syllables per word (Kincaid et al., 1975, p. 14):

$$\text{grade level} = 0.39 \times \frac{\text{words}}{\text{sentences}} + 11.8 \times \frac{\text{syllables}}{\text{words}} - 15.59.$$

It can therefore neither see meaning preservation nor distinguish a well-simplified sentence from a truncated one. BLESS makes the point empirically: the five models it ranks best by FKGL produced the least simple and least meaning-preserving outputs of any group it examined, hallucinating at 36 % against 12 % across all outputs (Kew et al., 2023, p. 13297). FKGL is reported throughout, but no claim in this thesis rests on it alone.

Document-adapted metrics exist because SARI has no notion of document structure. A document-level system can therefore score well on SARI while producing an implausibly short or implausibly fragmented document. Document-level SARI (D-SARI) (Sun et al., 2021, pp. 8001–8002) adapts SARI for exactly this, adding two length-penalty terms and a sentence-count penalty. These penalties discount the keep, add, and delete components when the output’s length or segmentation departs from the reference’s. Its sensitivity is structural only: the sentence-count penalty is a segmentation penalty rather than a cohesion measure, so it cannot see whether a pronoun resolves or a connective survives.

Learned and semantic metrics score outputs through learned representations rather than n -gram overlap. Learnable Evaluation metric for Text Simplification (LENS) (Maddela et al., 2023, pp. 16385–16386) is a supervised metric for the task, trained directly on human simplification ratings. It was trained on sentence-level ratings, so applying it to documents is out of domain by construction (Chapter 8). BERTScore (T. Zhang et al., 2020, p. 1) measures contextual-embedding similarity and is a general text-generation metric rather than a simplification-specific one.

Human evaluation is what all of the above are proxies for. It is elicited along three standard dimensions: grammaticality (often called fluency), meaning preservation (often called adequacy), and simplicity (Alva-Manchego, Scarton, & Specia, 2020, p. 147). No such study is conducted in this thesis. Chapter 9 proposes one, with a specific question and a minimum useful design.

2.5 Web pages as input

A web page is a structured document: its content sits in paragraphs, headings, lists, and tables, alongside regions such as navigation, headers and footers, and form controls with their labels (Hellbusch, 2018, pp. 518–519). Screen readers navigate this structure directly, for example by jumping from heading to heading (Hellbusch, 2018, p. 519). A simplifier applied to such a page receives text units whose boundaries are set by the markup rather than by sentence punctuation, and it must write its output back without disturbing the structure the page and its readers rely on. None of the corpora of Section 2.3 contains input of this kind. RQ2 addresses this gap, which motivates Hypothesis H_4 .

3 Related Work

3.1 Approaches to text simplification

The comparison simplifier is an off-the-shelf checkpoint¹ already fine-tuned for simplification, against which the model fine-tuned here is evaluated, a core objective of RQ1. Its architecture is BART-large (Lewis et al., 2020, p. 7872) at 406M parameters (Cohen et al., 2026, p. 2). It was fine-tuned not by BART’s original authors but by Cohen et al. (2026, pp. 1, 4), who released it with their comparative study. Its training data is the pre-cleaned WikiLarge mirror² this project also trained on, with the prefix **Simplify:** added to every source sentence (Cohen et al., 2026, pp. 3–4). The mechanism is plain supervised fine-tuning, with no control tokens, no auxiliary objective, and no reranking.

Three limitations affect the evaluation for RQ1: Because it shares its training corpus with this project’s sentence model, which additionally dropped mojibake rows (Section 4.2), a fractional difference in SARI between the two says little about either training run (Section 7.1). Because this project passes inputs without that prefix, the comparison simplifier was scored on inputs formatted differently from its training data (Chapter 8). And because it was fine-tuned on isolated sentences, it has no document counterpart, so it appears at one granularity only.

Controllable simplification with control tokens trains a model to follow an explicit request. ACCESS (Martin et al., 2020, p. 4691) is the canonical instance, and its successor Multilingual Unsupervised Sentence Simplification (MUSS) (Martin et al., 2022, p. 1653) is the baseline BLESS itself uses (Kew et al., 2023, p. 13293). The architecture is a seq2seq model, in MUSS’s case BART-large. The training setup conditions each training pair on attributes computed from the pair itself, such as compression ratio, paraphrase amount, and lexical and syntactic complexity. The model thus learns to follow a request instead of producing one fixed style, and at inference the attributes are set to the values the output should have.

¹[eilmc14/bart-large-text-simplification](#)

²[eilmc14/wikilarge-clean](#)

The limitation that matters here is that the control tokens are proxies (Martin et al., 2020, p. 4691): the model is conditioned on a measured attribute of the source–target pair, and nothing at inference checks that the property was achieved in the output. The prompted LLM approach of this thesis asks for control through the audience parameter instead of training it in. It inherits that limitation: nothing in this thesis tests whether the audience parameter helps the audience it names.

Prompted LLMs are represented here by BLESS (Kew et al., 2023, p. 13291), from which the third approach of this thesis takes its prompt. It evaluates 44 models, open- and closed-weight, differing in size, architecture, pretraining methods, and accessibility. The setup is few-shot in-context learning on three test sets from different domains, namely Wikipedia, news, and medical texts, with three stylistically distinct prompts. Each prompt carries three demonstrations drawn at random from the dataset’s validation split (Kew et al., 2023, p. 13293). Outputs are sampled (nucleus sampling with $\text{top-}p = 0.9$, temperature 1.0, at most 100 tokens) under three random seeds (Kew et al., 2023, p. 13293). No model is fine-tuned; the task is given only through the prompt. BLESS’s best models perform comparably with dedicated trained simplification baselines (Kew et al., 2023, p. 13291). A prompted model is therefore a demanding comparison for the 139M-parameter model fine-tuned here.

Two limitations of BLESS matter here. It is sentence-only, so its results do not cover documents. It leaves the study of in-context learning strategies for simplification to future work (Kew et al., 2023, p. 13293), which is why the demonstrations here are fixed (Section 4.4).

Document-level simplification supplies both the corpus and the metric on which the document results rest. Sun et al. (2021, p. 7999) assembled D-Wikipedia from aligned English and Simple English Wikipedia articles, keeping only the lead section of each. The authors evaluated it with four baselines (a vanilla Transformer, the context-conditioned sentence simplifier SUC, BertSumextabs, and BART) (Sun et al., 2021, p. 8003). They also introduced D-SARI to score it (Sun et al., 2021, p. 8002).

Like the fine-tuned document checkpoint here, SimDoc (Vásquez-Rodríguez et al., 2024, pp. 1, 3) fine-tunes a pretrained seq2seq model on document pairs. SimDoc uses Text-to-Text Transfer Transformer (T5), fine-tuned on professionally created simplification corpora extended into (complex text, simple text, readability label) triples. A combined loss joins simplification, readability classification, and a coherence reward, steered by task tokens in the input. The models fine-tuned in this thesis have no coherence objective (Chapter 9).

3.2 Browser-based deployment

The closest deployed system to this one is the on-device Firefox simplifier of Romero et al. (2025, p. 105). It runs BART-base (Romero et al., 2025, p. 107), quantised for in-browser execution. It is deployed through `transformers.js` so that inference runs inside the browser process itself. The model is trained on a synthetic corpus of 2,909 examples generated by prompting LLMs, which its authors report as outperforming the 300k-example WikiLarge corpus on standard benchmarks (Romero et al., 2025, p. 105). A single control token sets the target reading level, in place of the multi-token ACCESS scheme. However, separate models trained for one reading level each outperform that control-token model (Romero et al., 2025, p. 110). The authors find that quantisation reduces model size by up to 75 % with little performance degradation (Romero et al., 2025, p. 105), so a simplification model may be served entirely on a user’s device.

Unlike the Firefox simplifier, the extension built here calls a local backend service instead of running the model in the browser process. Both arrangements keep inference on the user’s own machine, which is the axis the privacy argument of Section 5.1 turns on. Romero et al. (2025) report quality on standard sentence benchmarks, so their work makes no claim about behaviour on authentic pages, which is the question RQ2 asks.

3.3 Behavioural robustness beyond held-out accuracy

The deployment half of this thesis takes its framing from CheckList (Ribeiro et al., 2020, p. 4902), a behavioural testing methodology. It borrows from behavioural testing in software engineering, takes as its input a task-agnostic matrix of linguistic capabilities crossed with test types, generates many test cases from templates and lexicons, and reports a failure rate per test.

It distinguishes three test types (Ribeiro et al., 2020, pp. 4902–4903): minimum functionality tests for a targeted behaviour, invariance tests for perturbations that should not change the prediction, and directional expectation tests for perturbations whose effect is known in advance. Held-out accuracy is the standard measure of generalisation, but it often overestimates real performance, because held-out sets often carry the same biases as the training data (Ribeiro et al., 2020, p. 4902). Ribeiro et al. (2020, pp. 4906, 4910) present CheckList as a complement to benchmarks, not a replacement; passing its tests does not imply that a model is free of the tested failure.

The site audit of Section 6.6 is a robustness study in that sense. The deployment adds a structural constraint to the linguistic ones: a simplification system that edits a live page must write text back into a document whose markup, links, attributes, and interactive elements survive the edit.

3.4 Research hypotheses and expectations

Four hypotheses follow from the work reviewed above: two on the comparisons that the design of Chapter 4 sets up (H_1 , H_2), one on how the metrics rank the compared outputs (H_3), and one on the deployment (H_4). All four assume that findings reported for other systems carry over to the ones used here, and each is stated with the result that would refute it.

- H_1** Fine-tuning a pretrained seq2seq model on task-matched parallel data improves simplification-specific metrics over the same checkpoint before fine-tuning, the one contrast in the grid that holds architecture and scale fixed. This is the premise of the fine-tuned systems of Section 3.1, among them MUSS, which fine-tunes BART on mined paraphrases and, in its supervised variant, on labelled simplification data (Martin et al., 2022, pp. 1652–1653). H_1 fails if the paired bootstrap finds the SARI or the D-SARI improvement not significant at $p < 0.05$.
- H_2** A prompted instruction-tuned LLM of a recent generation matches or exceeds a fine-tuned model, since BLESS finds that its best models perform comparably with trained simplification baselines (Kew et al., 2023, p. 13291). At each granularity, H_2 holds if a prompted model scores at or above the fine-tuned checkpoint on every metric compared there (SARI on ASSET at sentence level; D-SARI and LENS on D-Wikipedia at document level), fails if every prompted model scores below it on all of them, and remains open if the metrics disagree.
- H_3** Reference-overlap and simplification-specific metrics rank the same outputs differently, because BLEU leaves latitude for matching the input while SARI scores additions, kept n -grams and deletions against both the source and the references (Xu et al., 2016, pp. 404–405). H_3 fails if BLEU and SARI order the five sentence-level conditions the same way.
- H_4** Behaviour on authentic web pages diverges from benchmark behaviour, since held-out accuracy often overestimates real performance (Ribeiro et al., 2020, p. 4902) and web pages supply units that no benchmark contains (Section 2.5). H_4 is tested qualitatively: it fails if every failure class the site audit finds also appears in the benchmark evaluation.

How the hypotheses are tested. H_1 and H_2 are tested across the sentence-level configurations and document-level runs detailed in Chapter 4. H_3 is evaluated on the five sentence-level conditions within the same chapter. Finally, H_4 is tested via the deployment site audit described in Chapter 5.

4 Methodology I: Simplification Approaches

4.1 Comparison design

The comparison is built as a grid of three approaches against two granularities. The approaches are the comparison simplifier, a seq2seq model fine-tuned for this project, and an open-weight instruction-tuned LLM steered by a prompt; the granularities are the sentence and the document.

To answer RQ1, the trained model is compared with two checkpoints.

1. The **untuned checkpoint** is the BART-base model before fine-tuning.
2. The **comparison simplifier** is the off-the-shelf third-party fine-tuned model described in Section 3.1. It is roughly three times as large and was trained on the same WikiLarge corpus.

The fine-tuned model on sentence-level and the comparison simplifier therefore differ mainly in model size and in their fine-tuning runs, which were set up independently. There is no document counterpart, since the comparison simplifier was fine-tuned on single sentences (Section 3.1).

4.2 Corpus assessment and preparation

Table 4.1 specifies the five assessed corpora and the decision taken on each.

The WikiSmall distribution¹ was rejected on inspected evidence. Throughout its text named entities are replaced by placeholders such as `PERSON@1` and `LOCATION@1`. A model trained on such pairs would learn to produce placeholders where a page has names. The placeholders appear to be a property of this distribution rather than of the original corpus: Zhu et al. (2010, p. 1354) describe no named-entity replacement step for PWKP, the corpus distributed as WikiSmall. They most likely come from the release of X. Zhang and Lapata (2017, pp. 588–589), who replaced named entities with tokens such as `PER@1` and whose WikiSmall training set has 89,042 pairs, the size of the distribution inspected here.

¹cestwc/adapted-wikismall

Table 4.1
Corpus Assessment and Selection Decisions

Corpus (unit)	Rows as used here	Convention and alignment	Decision
WikiLarge (sent. pairs)	121,657 / 410 / 121 filtered; 123,862 / 417 / 121 raw; source P_{90} ≈ 52 tokens	natural case; doubly-encoded characters in a minority of rows; automatic, aggregates earlier collections, among them PWKP	Adopted for sentence-level training
WikiSmall (sent. pairs)	≈ 89 k pairs in the distribution inspected; splits unused	named-entity placeholders throughout (PERSON01); automatic, sentence-level term frequency-inverse document frequency (TF*IDF)	Rejected on named-entity anonymisation
D-Wikipedia (doc. pairs)	131,739 train (filtered); 8,000 test; mean 180, median 120, P_{95} 510, max 1,168 tokens	lowercased, Penn-Treebank pre-tokenised, one document per line; automatic	Adopted for document-level training
SWiPE (doc. pairs)	≈ 145 k pairs; splits unused	automatic, with human edit-category annotation	Rejected ; built for classifying edit types, not for plain training
ASSET (sent., multi-ref.)	no train split; validation used for demonstrations; 359 test	natural case; human rewrites, ten per source sentence	Adopted for sentence-level evaluation only

Note. All corpora are English only; counts describe the distributions used here, not the corpora as published: X. Zhang & Lapata (2017, p. 589) report 296,402 pairs for WikiLarge. Lengths are source lengths in tokens; D-Wikipedia’s come from a sample of 1,500 source documents, and its validation split and the two rejected corpora were not profiled. Sources, in table order: (X. Zhang & Lapata, 2017; Zhu et al., 2010; Sun et al., 2021; Laban et al., 2023; Alva-Manchego, Martin, et al., 2020).

SWiPE² was rejected as built for a different question. It pairs English and Simple English Wikipedia pages reconstructed from their revision histories, with a subset of about 5,000 pairs annotated for 19 categories of edits (Laban et al., 2023, pp. 10674–10675). That annotation serves the classification of edit types, whereas flat document pairs were needed for the seq2seq training.

²salesforce/simplification

A pre-cleaned WikiLarge mirror was adopted, but it required further cleaning. The mirror is the one introduced in Section 3.1; its curator had already deduplicated it and filtered it by token length, compression ratio, and lexical similarity (Cohen et al., 2026, pp. 3–4). One defect survived that filtering: mojibake, the doubly-encoded byte sequences produced when UTF-8 text is decoded as Latin-1 somewhere in an alignment pipeline. Affected rows are dropped rather than repaired, since the standard repair tool, `ftfy`, does not recognise this pattern: the broken byte sequences are no longer contiguous.

Two filters exist (Table 4.2). A strict filter drops any row containing a bare `â` or `Ã`, while a targeted one matches seven specific broken sequences. The strict filter drops every row the targeted one drops, plus 4,001 further training rows (3.3%) that inspection identified as mojibake the targeted patterns miss. A spot check found legitimate accented text surviving the strict filter. The deployed sentence checkpoint was trained under the targeted filter, while the corpus statistics in the repository’s data statement (`research/data/README.md` and `stats.json`) are those of the strict filter.

Table 4.2
Comparison of WikiLarge Split Sizes Under the Two Mojibake Filters

Split	Raw	Strict filter	Targeted filter
train	≈123k	≈117k (kept 95.0 %)	≈121k (kept 98.2 %)
validation	417	397 (kept 95.2 %)	410 (kept 98.3 %)
test	121	121 (kept 100 %)	121 (kept 100 %)
all splits	≈124k	≈118k (kept 95.0 %)	≈122k (kept 98.2 %)

Note. Percentages are of the raw split. Exact training-row counts are 123,862 raw, 117,656 strict and 121,657 targeted.

D-Wikipedia needed its own loader. It is distributed not through the HF Hub but as raw files from the paper’s own repository, with the training split in a compressed archive. The cleaning code was reused unmodified, since it operates on whole strings regardless of whether a string is one sentence or one article. The two granularities’ corpora were therefore prepared identically, additionally they supply their pairs in the same flat source–target form.

Three scope tiers bound the cost of each run. The pipeline defines them at both granularities: 200 rows (`prove_loop`), 20,000 rows (`reduced`) and the entire cleaned split (`full`), so that every configuration runs on 200 rows before compute is spent on the full split.

4.3 Fine-tuning sequence-to-sequence models

Both fine-tuned checkpoints start from BART-base (Lewis et al., 2020, p. 7872) and were trained with the `Seq2SeqTrainer` of the Transformers library (Wolf et al., 2020). Their data collator sets `label_pad_token_id=-100`, which excludes padding from the loss. The two granularities share the entire pipeline (tokenisation, trainer setup, decoding, and significance testing), and a single flag selects the dataset structure. Differences between the granularities’ results should therefore come from the data, not from the implementation.

Early stopping after four evaluations without improvement was enabled, as was selection of the best checkpoint by validation loss. The trainer evaluated every quarter epoch and saved a checkpoint once per epoch. The sentence-level run shows the selection working, since epoch 4 was selected over epoch 5 (0.4568 against 0.4590). Because evaluation and saving run at different intervals, the best validation loss the trainer records may fall at a step for which no checkpoint exists. In that case, the figure quoted is the one the selected weights record. Since 19.02 % of the WikiLarge validation split also appears in the training data (Section 6.1), the validation losses reported here are training diagnostics, not estimates of generalisation. Checkpoint selection is unaffected, because every checkpoint is scored on the same validation split.

The length limits follow the training setup and the measured document lengths. The sentence models generate with `max_length=64`, the value their fine-tuning used, which aligns inference with training at the cost of cutting off any output longer than 64 tokens. The 90th percentile (P_{90}) of the source length is ≈ 52 tokens. The document models use an input cap of 512 tokens, chosen from the measured token-length distribution of the training corpus under the available memory budget (Figure 4.1). BART’s positional limit is 1,024 tokens. Section 6.7 uses the same distribution to decide whether page sections are merged into one document input.

The training configuration depends on the device (Table 4.3). On Apple Silicon, dynamic per-batch padding let reserved memory grow past 13 GiB within roughly a hundred steps, against ≈ 4.5 GiB with fixed-length padding. Sentence training on Apple Silicon therefore uses fixed padding, gradient accumulation, and gradient checkpointing. At document level, the progress estimate of a full-scope run exceeded the session cap of the hosted Google Colab tier (Google Research, 2026), the full-scope run therefore was completed on an RTX A5000 in under 3 h.

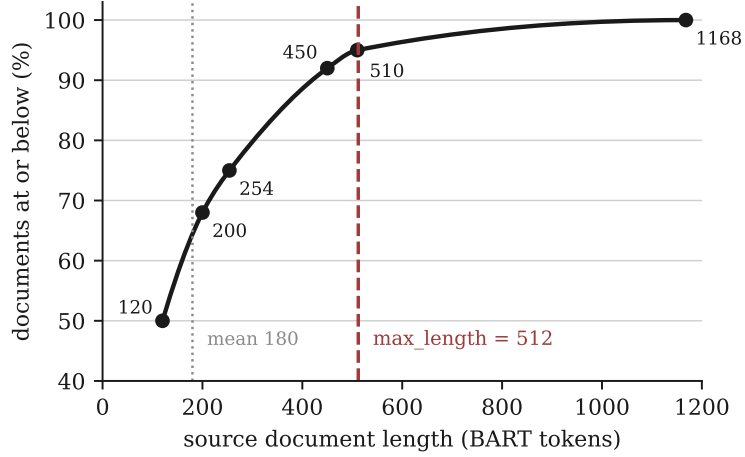


Figure 4.1

Source-Document Length Distribution Relative to the 512-Token Input Cap

Note. The six markers are measured quantiles; the curve joining them is a monotone interpolation, not measured data.

Table 4.3

Training Configuration by Granularity, Scope, and Device

Granularity; scope	Rows	Epochs	Batch	max_len	Device
sentence; <code>prove_loop</code>	200	1	8	64	MPS
sentence; <code>reduced</code>	$\approx 20k$	2	16	64	MPS
sentence; <code>full</code>	$\approx 121k$	5	16	64	CUDA
document; <code>reduced</code>	$\approx 20k$	2	8	512	CUDA
document; <code>full</code>	$\approx 131k$	5	32	512	RTX

Note. Bold marks the two deployed checkpoints (exact row counts: Table 4.2). Devices are Metal Performance Shaders (MPS) on Apple Silicon, CUDA on a hosted Google Colab T4 in 16-bit floating point (fp16), and RTX on an RTX A5000 in bfloat16 (bf16). All runs share a learning rate of 3×10^{-5} , weight decay 0.01, seed 42, early stopping with patience 4, and best checkpoint by validation loss.

Decoding is by beam search, and the checkpoints are published. All reported seq2seq numbers use beam search (`num_beams=4`, `no_repeat_ngram_size=3`, `repetition_penalty=1.2`). Beam search explores several candidate continuations in parallel instead of committing to the locally most probable token at each step (Tunstall et al., 2023, pp. 224–225). One early run was logged as *greedy* but passed no decoding arguments, so it used the checkpoint’s own four-beam configuration. A true greedy run was added afterwards, and the three runs span 0.26 SARI (Section 6.2).

Three checkpoints are published on the HF Hub, with model cards recording configuration, metrics, and limitations. At sentence level, the published checkpoint³ is the deployed sentence checkpoint. At document level, both the reduced-scope⁴ and the full-scope⁵ checkpoints are published. The full-scope checkpoint is the one the document results report and the backend serves.

4.4 Prompted open-weight language models

Only open-weight LLMs were considered. A hosted application programming interface (API) would void the local-processing constraint of Section 5.1. The constraint has a known cost, since Kew et al. (2023, p. 13291) report that closed-weight models provide significant gains over open-weight ones under few-shot prompting. A local Ollama server (Ollama, 2026a) serves the model in 4-bit quantised GGUF form, a model file format Ollama imports (Ollama, 2026b). At roughly 4.7 GB, the quantised 7B model fits alongside the BART checkpoints on a 16 GB machine, which the roughly 14 GB half-precision version would not. Two instruction-tuned models of the Qwen2.5 family (Qwen et al., 2025, p. 1) were evaluated, one with 7B⁶ and one with 3B⁷ parameters. The backend sends the prompted model one request at a time, so the prompted approach runs at a batch size of one. It does not use the batching the seq2seq checkpoints rely on, which may have limited its throughput (Section 6.8).

The sentence prompt is BLESS Prompt 2 with an audience slot. It reproduces the prompt verbatim (Kew et al., 2023, p. 13294), with one change: the hard-coded audience *non-native speakers of English* becomes a slot that holds one of five audience values, such as *children aged 8 to 12*. The original phrase is the default. Appendix A.2 gives the exact template, the audience values, and the decoding settings.

³`yunvs/bart-base-wikilarge-simplification`

⁴`yunvs/bart-base-dwikipedia-simplification`

⁵`yunvs/bart-base-dwikipedia-simplification-full`

⁶`qwen2.5:7b-instruct-q4_K_M`

⁷`qwen2.5:3b-instruct-q4_K_M`

ASSET’s references were written under instructions for non-native speakers of English, the instructions BLESS’s Prompt 2 repurposes (Kew et al., 2023, p. 13294). Pairing them with an instruction to simplify for children would set the demonstrations against the instruction. Each audience therefore has its own demonstration pool: ASSET pairs for the default audience and hand-authored pairs for the other four (Appendix A.3).

The default demonstrations are fixed and pass four selection rules. BLESS samples its demonstrations at random and leaves the study of in-context learning strategies open (Section 3.1). Here, the same three demonstrations are used in every run: ASSET validation indices 285, 1516, and 1116 (Appendix A.3). They were selected by four rules. The first is a random draw at seed 42; the other three are filters, each added after an observed defect:

- the source has at least 20 words, so that at least one demonstration can show a sentence split;
- the reference is well-formed, which excludes pairs with crowdworker punctuation typos;
- at least half of the reference’s content words occur in the source.

The last filter excluded an ASSET reference that is fluent and well punctuated but has a different meaning from its source, the kind of misalignment Xu et al. (2015, p. 285) found in 17 % of 200 sampled PWKP pairs. Together, the three demonstrations show all three operations the instruction names (paraphrasing, compression, and splitting). None of the three demonstrations occurs in the WikiLarge training data.

Failed outputs are classified by reason code. The output check of the prompted approach, one of the guards described in Section 5.4, returns a *reason code* instead of a pass or fail. Some codes cover failures typical of a prompted LLM, such as prompt echo and refusal. The others cover failures any generator can produce: empty output, output without words, degenerate repetition, over-long output, unrelated short output, timeout, and error.

The counts per code feed the error categorisation directly. When the check fails, the original text is kept, so a bad output leaves the sentence unsimplified. Output identical to the input does not count as a failure: copying the source is a documented behaviour on this task (Kew et al., 2023, p. 13298). Such output is therefore measured and reported per condition as the unchanged rate.

The document prompt adapts the sentence prompt. BLESS is sentence-only, so what carries over is the prompt’s structure. Appendix A.2 prints the adapted template beside the original. Three differences are recorded: merging and reordering are added to the permitted operations; the output is constrained to continuous prose with no preamble, headings, or bullet points; and the frame is `Text:/Simplified text:` instead of `Complex:/Simple:`. The document prompt carries no demonstrations by default: each would be a whole document pair, which would make the prompt much longer. Such pairs would moreover come from D-Wikipedia, which is lowercased and pre-tokenised, unlike typical web-page text. The prompted approach therefore does not denote the same configuration at the two granularities.

Two decoding configurations are likewise reported separately: greedy decoding at temperature 0 for serving, so that re-simplifying a page is stable, and BLESS’s own sampling settings (Kew et al., 2023, p. 13293) for evaluation, so that the numbers remain comparable to the paper. The extension and the results table therefore do not describe the same system.

4.5 Evaluation protocol

Each granularity is scored on its own benchmark, but by the same evaluation code. Sentence-level conditions are scored on the ASSET test split (Alva-Manchego, Martin, et al., 2020, pp. 4670–4671). Document-level conditions are scored on D-Wikipedia’s own test split.

Scoring runs in a standalone evaluation pipeline, not in the metric code built into the trainer. SARI and FKGL are computed with EASSE, a toolkit for evaluating sentence simplification (Alva-Manchego et al., 2019, pp. 49–50). Every approach passes through the same data loader, the same reference set per benchmark, and the same metric calls, so differences between approaches cannot come from the scoring code.

Significance is tested by paired bootstrap resampling. Improvements in SARI and D-SARI are tested (Koehn, 2004, p. 393) with 1,000 resamples, applying the same resampled indices to both systems so that pairing is preserved; this provides the empirical basis for refuting Hypothesis H_1 . Where the fine-tuned system wins every resample, the bootstrap estimate is $p = 0$. This estimate is reported as $p < 0.001$, because 1,000 resamples cannot distinguish a truly zero probability from a very small one. LENS and BERTScore are reported descriptively; testing their differences for significance is outside the scope of this thesis.

Metric preprocessing is part of the metric. Where a metric carries structural penalties, its input preprocessing changes what it measures. Normalising into D-Wikipedia’s convention has no effect on SARI and FKGL, which re-tokenise their input anyway, and changes D-SARI, whose sentence-count penalty reads exactly the boundaries that normalisation creates or destroys. Section 6.3 measures this by rescoring cached generations under two versions of the normaliser.

Deployment inference and benchmark inference run in different places. Deployment inference runs locally, because privacy is a design constraint. Benchmark inference runs on an RTX A5000 graphics processing unit (GPU) instead. ASSET and D-Wikipedia are public datasets that contain no user data, so the privacy constraint does not apply to them. Local hardware was also too slow for the full five-condition evaluation, which the GPU completed in 42 minutes.

Provenance is recorded, and the D-SARI reimplementations are tested. From the first run on the RTX A5000 onwards, a staged runner committed to the repository produced every number on that machine, at one code revision. Before it spends any GPU time, the runner executes a correctness check, which caught a metric-normalisation defect before a training run. Every artefact records the software environment that produced it, including the exact commit of the SARI implementation.

A differential test compares the D-SARI reimplementations with the reference implementation: across 66 cases drawn from real D-Wikipedia documents, the two must agree to full floating-point precision. Nine further invariant checks run offline. Since the reference implementation ships no tests of its own, this test is what makes the document scores comparable with published D-SARI figures. The reimplementations keep two properties of the reference implementation, so that it measures D-SARI as published. Deletion contributes a precision term instead of an F-score, as in the definition of SARI (Xu et al., 2016, p. 405), and the reference implementation computes one intermediate delete-recall term that it never uses.

5 Methodology II: Browser Extension Deployment

To address the deployment challenges and practical robustness queried in RQ2, this chapter details the implementation of a local-processing framework. The resulting architecture is directly shaped by the models of Chapter 4, by their granularity, their input caps, and their failure modes, as well as by the privacy constraints that ruled out hosted models.

5.1 Architecture and local-processing constraints

The system has three components: a Chrome Manifest V3 extension (Chrome for Developers, 2023), a FastAPI backend (FastAPI, 2026), and the loaded models. The extension consists of a content script, which runs in the context of web pages (Chrome for Developers, 2012), and a service worker. Seven selectable keys sit on five checkpoints: the sentence and document checkpoints fine-tuned here, the comparison simplifier, and the two prompted Qwen2.5 models, each offered at both granularities.

Figure 5.1 gives the arrangement and Figure 5.2 one request’s path through it, from collecting candidate text to writing the output back. The response carries the whole chain: the input, what the model produced, what was served, and why the two differ. This chain is what makes the History log of Section 5.5 usable as evidence in Section 6.5.

The backend writes no data to persistent logs or storage and keeps only an in-memory cache, which is cleared on restart. The extension does store text: the History log keeps the last twenty pages in the browser’s local storage, on the user’s own machine. The backend binds to `127.0.0.1` (FastAPI, 2026) and is reachable only from the user’s own machine. None of the ATS approaches, including the prompted approach, sends data to a third party.

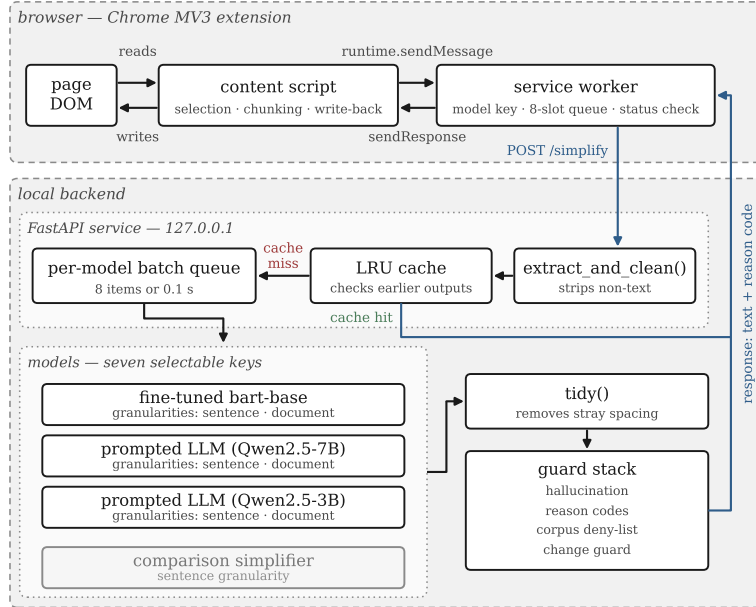
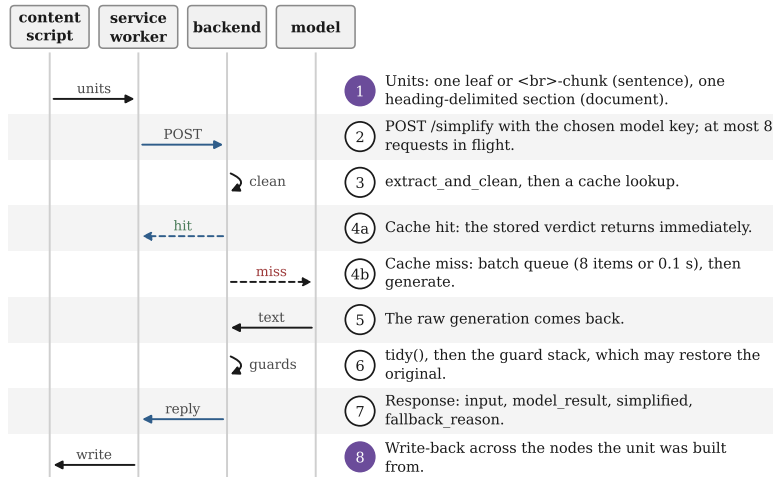


Figure 5.1

System Architecture and the Local-Processing Constraint

Note. All components run on-device; LLMs are served by a separate local Ollama server. The comparison simplifier is a third-party checkpoint not produced in this work.



Steps 1 and 8 are the only two points at which the sentence and document cuts differ.
 Blue: the HTTP boundary. Step 2 is the API call, steps 4a and 7 its two possible responses.
 Dashed: steps 4a and 4b are the two mutually exclusive outcomes of the cache lookup.

Figure 5.2

The Request Path, and Where the Granularities Diverge

5.2 Content selection and DOM-preserving replacement

The extension prepares a page in one of two ways, one per granularity. The *sentence cut* (or sentence mode) sends each prose leaf, which the backend splits into sentences. The *document cut* (or document mode) sends each heading-delimited section as one document (Section 5.3).

Candidates are classified as blocks or leaves, then filtered four ways. The system classifies candidate elements as block-like containers or inline leaf elements. A block-like element counts as a leaf only if none of its descendants is itself block-like, which prevents a wrapping `<div>`, or a table around its cells, from being sent twice.

Four filters then run: a minimum word count (`MIN_WORDS_TO_SIMPLIFY=4`), a requirement that the text contain letters, a skip for text that is only a uniform resource locator (URL), and a regular expression matching `/\b(function|var|let|const|=>)\b|[\{\}<>;=]/` that skips code-like text even outside `<pre>` and `<code>`. Elements inside `<pre>`, `<code>`, `<script>`, or `<style>` are excluded outright. The minimum was originally 30 characters, which would have excluded most buttons, labels and other short interactive elements. It was lowered to a four-word minimum. The lower threshold admits many more requests per page, but also increases the amount of possible candidates for simplification (Section 6.6).

Opted-out text is read as context but never rewritten. The extension treats three markers as opt-outs at element and text-node level: `translate="no"` (WHATWG, 2026), `class="notranslate"` (Google Search Central, 2008) and `aria-hidden` (W3C, 2026). Both cuts originally removed opted-out subtrees from the request entirely, and therefore the opted-out text wasn't sent to the backend. For the sentence cut this removal is valid, but not for the document cut, since the holes it left in a section's prose were filled with hallucinated content. Therefore, the extension now also sends opted-out text to the backend as context but doesn't run simplification on it.

Write-back is anchored on fragments the model left verbatim. A unit's output is written back into the text nodes the request was built from. Earlier designs preserved either formatting or sentences, but not both. Replacing each text node separately kept inline formatting but broke sentences into fragments; flattening an element and replacing its content kept sentences whole but removed links and formatting. Anchored write-back keeps both, provided that some fragments of the original survive verbatim in the rewrite, since the alignment anchors on them (Section 6.6).

In the document cut, the output does not line up with its input element by element, since the model may merge several elements into one. The output is therefore redistributed across the elements the section was built from: each sentence goes to an element by content-word overlap, or by each element's share of the input's length (*pacing*).

A page asserts structural boundaries that carry no punctuation. A `<br>` is a structural break the document asserts outright, and in the extension's text extraction it contributes neither characters nor punctuation to an element's text. A line such as `Tickets cannot be refunded<br>Children under 12 travel free` therefore flattens into a single run, which reads as one sentence that a punctuation-based segmenter cannot split accurately. The rule adopted is therefore two-stage and ordered: it splits on structural boundaries first and segments linguistically within each chunk afterwards. The reverse order would ask a punctuation-based segmenter to recover information that no longer exists. A blank line inside a text node under `white-space: pre-wrap` is the same defect in a second spelling.

Security rests on an API property, not on sanitising a string. Model output is untrusted input to the page, and it is assigned to a text node's data, which cannot create an element. A committed test asserts that an output containing `<img src=x onerror='alert(1)''>` renders as visible escaped text.

The original text is stored on the page itself. Before an element is rewritten, its original content is stored in its `data-original-*` HyperText Markup Language (HTML) attributes. These attributes mark the element as already simplified, so that a repeated run leaves it alone, and they allow the page to be restored. A reload discards these attributes together with the simplification, and the page appears as its host served it. Simplifying it again then reuses earlier work: the backend serves its cached output for any unit it has already processed with the same model and audience.

5.3 Unit of simplification

The originating defect was fragmentation. Running the extension on an English Wikipedia article produced output in which single sentences had been broken into several independently simplified pieces: one article contributed 173 units, including fragments such as `") is a nonprofit "` and `", later known as the "`. The cause was that every text node was sent separately and any inline element terminates a text node, so a sentence with two links and a footnote marker is split into four or more fragments.

Per-text-node replacement was a deliberate feature, since it is exactly what preserves inline formatting, an explicit design goal. The design optimised for DOM integrity and thereby destroyed linguistic integrity (Section 6.7).

The sentence split belongs in the backend. It was placed there because it is one implementation serving every client, and because the split is a property of the model and not of the page. The prompted approach’s single-sentence template was being degraded by flattened multi-sentence paragraphs independently of anything in the DOM. A rule-based sentence segmenter, `pysbd`, was chosen over a single regular expression. A regular expression could split sentences wrongly, because a full stop can end a sentence, be used as a decimal point, or mark an abbreviation (Jurafsky & Martin, 2026, pp. 59–60). Granularity thereby becomes an explicit per-model property that the backend’s status endpoint reports, because two models can share an architecture and still require different text units.

A paragraph is not a document. The document model was trained on documents averaging roughly 180 tokens, with a median of 120. Sending it one paragraph at a time would create a fragmentation problem. Like a sentence model given sentence fragments, the document model would receive a smaller unit of text, which it essentially was not trained for. Sending a whole web page as one document was rejected as well, since it would have silently deleted the greater part of most articles at the 512-token input cap.

The chosen design therefore determines the page’s content area and collects prose leaves in document order until it reaches the next heading or the model’s input cap, which the extension reads from the backend’s status endpoint. A section longer than the input cap is split into segments that are sent as separate requests, and their outputs are rejoined before insertion (Table 5.1).

Table 5.1
Segment-Join Strategies and Their Failure Modes

Strategy	Result
join with a space	adjacent segments fuse into one sentence, colliding unrelated facts
append a full stop unconditionally, then join	double punctuation after segments already ending in terminal punctuation
append a full stop only where one is missing, then join with a space	adopted ; boundaries survive and no punctuation is duplicated

Headings themselves are not sent or rewritten, since including them in a section’s input resulted in them being echoed at the start of the output (Table 5.2). Additionally, leaving them untouched preserves the page’s heading structure, which screen readers use to navigate (Hellbusch, 2018, p. 519). A strategy that would merge short sections up to the 512-token input cap was planned but dropped, since long combined sections were not as represented in the training distribution.

Table 5.2
Effect of Input Convention on Document-Model Output

Input convention	Outcome
corpus convention (lowercased, pre-tokenised, no newlines)	two of the three facts retained, no fabrication; the adopted convention
ordinary mixed-case prose	a factual hallucination (“northern Netherlands” became “northern hemisphere”)
corpus convention with <code>\n</code> separators	two facts were lost, and the province “Friesland” became “the country of friesland”
heading text included in the section body	the heading echoed back at the start of the output, which would duplicate it on the page

Note. Outcomes were judged by inspecting one hand-constructed document ($n = 1$) for hallucination and for facts retained or lost. Appendix A.8 gives the recorded outputs.

The corpus convention becomes an inference-time variable. Because the document model both consumes and produces the corpus’s lowercased, pre-tokenised convention, the fix is a matched pair: normalise on the way in, de-normalise on the way out. The output half is needed because raw output such as `achtkarspelen is a municipality in friesland` would otherwise reach the page in lowercase.

This also matters for structural correctness: a single missing space after a full stop in the de-normaliser shifted an entire section’s write-back one position out of step, mis-assigning nine consecutive paragraphs and making one sentence disappear. The input convention was chosen by comparing the corpus convention with alternatives on one document and reading the outputs (Table 5.2).

5.4 Output guards and execution control

Four guards check generated text before it is served; rejected outputs are skipped and replaced by their original input. For the sentence-level seq2seq models, a word-overlap guard treats an output as a hallucination when the input has at most three words and the output shares none of them. It was built after the implausible outputs for isolated fragments described in Section 1.1.

For the two prompted models, the output check of Section 4.4 assigns a reason code to each failed output, such as prompt echo, refusal, empty output or degenerate repetition.

For all approaches, a change guard rejects output that differs from its input only in punctuation (e.g., a straightened quotation mark or a dropped full stop) or only by a deleted number. Its reason code is `no_meaningful_change`. Word-level edits are never rejected, and a punctuation-only edit is kept if it lowers the FKGL grade by at least half a grade level.

Also for all approaches, a deny-list rejects output that consists only of a known training-corpus heading; its two entries are “other websites” and “other pages”. Section 6.6 reports how often the guards fire, not whether each firing was correct.

In the document cut, an output with fewer sentences than its section has elements leaves the remaining elements unchanged. The extension records how many elements each output covered but does not close the gap.

Only one simplification run may proceed at a time, a precondition for the latency measurements. Two pages simplified concurrently would share the backend and its batch queue for each model, so their requests would interleave and the latency each page reports would describe neither run accurately.

5.5 Instruments for evaluating deployment

Three instruments produce the deployment evidence: the extension’s own History log, a latency profiler and a site-audit harness. The profiler and the harness run the extension’s content scripts. Like the behavioural testing of Section 3.3, all three examine how the deployed system behaves on real input rather than how it scores on a held-out benchmark.

The **History log** records the input, the model’s output and, where one applies, the guard reason, for every unit sent to the backend. Each run also records the approach, model identifier, granularity, and audience. It was built as a transparency feature and became the diagnostic instrument that identified the fragmentation defect.

The **latency profiler** runs the extension’s content script under `jsdom`, a JavaScript implementation of the DOM and HTML standards for Node.js (jsdom contributors, 2026), and sends its requests to the backend. The messaging between the content script and the service worker is replaced by a stub, so the requests are produced by the extension’s own code. The backend runs without automatic reloading, which would restart it whenever its code changes. The backend is restarted before each cold run so that no outputs are cached from an earlier run over the same page.

The **site-audit harness** runs every content script the extension manifest injects, under `jsdom` against the backend. It records the page’s anchors, images, citation markers and the count of each tag before and after a run, as well as the reason an element was skipped. It then reverts the page and compares the result with the original.

The extension also offers an **in-page multimodel comparison**, which runs one page through several models at a fixed granularity. It collects the page’s text once and sends the same inputs to every model, so the content cannot change between models. It helps users to compare and choose among the served models. Reported metrics come from the single evaluation harness of Section 4.5.

Committed `jsdom` and backend test suites cover the deployment path. They check separately that the right units are sent and that the outputs are written back correctly, since a suite covering one says nothing about the other. Tests that assumed the sentence cut’s behaviour exposed three development defects in the document cut: it folded a section’s paragraphs into one, recorded the output text as the original input, and did not correctly count the elements it passed over. All three were fixed. During development, the extension was also checked by hand in Chrome on several pages, including pages rendered entirely by client-side JavaScript.

6 Results

6.1 Corpus assessment

Cleaning WikiLarge under the strict mojibake filter drops 5.0 % of training rows (6,206 of 123,862), 4.8 % of validation rows, and no test rows. The targeted filter drops 1.8 % of training rows and 1.7 % of validation rows (Table 4.2). The deployed sentence checkpoint was trained on the targeted-filter corpus.

The overlap audit matches sentences exactly after Unicode normalisation, so its counts are lower bounds. The WikiLarge mirror’s 121-sentence test split is a 100 % subset of ASSET’s 359-sentence split, since both draw on the 359 test sentences of TurkCorpus (X. Zhang & Lapata, 2017, pp. 588–589; Alva-Manchego, Martin, et al., 2020, p. 4671). An improvement that appears on both test sets therefore shows that the scores agree on the same sentences, not that the model generalises to new ones. 19.02 % of the WikiLarge validation split also appears in training, so every validation loss here is optimistically biased.

Eight of ASSET’s 359 test sentences (2.23 %) also appear in the WikiLarge training data. Without them, every condition’s SARI changes by less than 0.12, and the gain of the fine-tuned over the untuned checkpoint changes by +0.006. None of the three demonstrations in the LLM prompts appears in the training data.

6.2 Sentence-level results

Table 6.1 is the primary sentence-level result: five conditions on the ASSET test split, all on one device at one code revision.

Against the untuned baseline checkpoint, fine-tuning improves SARI by +16.42 points at $p < 0.001$, with a confidence interval (CI) excluding zero by more than fifteen points, and improves FKGL by 1.84 grade levels. Against the comparison simplifier the difference is -0.37 SARI, with FKGL likewise marginally in favour of the comparison simplifier. No paired test between the two was run, and their intervals against the untuned checkpoint overlap almost entirely.

Table 6.1*Sentence-Level Comparison of the Five Conditions on ASSET*

Model	Params	SARI	BLEU	FKGL	Unchanged	Δ SARI 95 % CI
A: untuned	139M	21.39	91.64	10.02	93.9 %	n/a
B: fine-tuned	139M	37.80	88.30	8.18	11.7 %	[+15.38, +17.50]
C: comparison	406M	38.17	88.58	7.85	13.4 %	[+15.77, +17.82]
D: LLM (7B)	7B	45.95	75.46	7.77	4.5 %	[+23.64, +25.70]
E: LLM (3B)	3B	47.03	68.21	8.22	1.2 %	[+24.80, +26.51]

Note. $n = 359$ sentences with ten references each. Bold marks the best value per column, highest except for FKGL; BLEU is a counterexample, not a quality ranking (Section 2.4). **A–C** use beam search; **D** and **E** are three-shot over three seeds, reported as means (SARI standard deviations 0.073 and 0.053). Δ SARI intervals are the paired bootstrap over 1,000 resamples against **A**.

Both prompted models score above the fine-tuned model and the comparison simplifier, and the smaller of the two scores highest: $47.03 > 45.95 > 38.17 \approx 37.80 > 21.39$. The 3B-over-7B ordering is unlikely to come from decoding randomness: across three seeds each, the SARI standard deviations are 0.053 and 0.073, so the 1.07-point gap is more than 14 standard deviations. No paired test over sentences was run.

BLEU runs inverse to SARI across the five rows except for **B** and **C**, which are 0.37 SARI apart (Figure 6.1a). The same run measures the mechanism (Figure 6.1b): the unchanged rate falls from 93.9 % to 1.2 % as SARI rises, with the same exception. This pattern is the behaviour Section 2.4 predicts of a metric scoring n -gram overlap with a reference (Papineni et al., 2002, p. 312). A second explanation, that ASSET’s ten references amplify a conservative system’s advantage, does not hold. Sweeping the reference count from $k = 1$ to $k = 10$ (Figure 6.2) lifts the absolute BLEU of the untuned and the fine-tuned checkpoint steeply while leaving the gap between them at two to three points throughout. Reference count therefore does not explain the inversion; the edit rate remains the explanation consistent with Figure 6.1b.

The noise floor is ≈ 2 SARI points, estimated from two unseeded reduced-scope training runs scored on 50 WikiLarge test sentences (35.85 and 33.85). The deterministic untuned checkpoint scored identically in both. The estimate is a rough bound for the full configuration rather than a measurement of it. The three decoding runs of Section 4.3 span 0.26 SARI, well inside it.

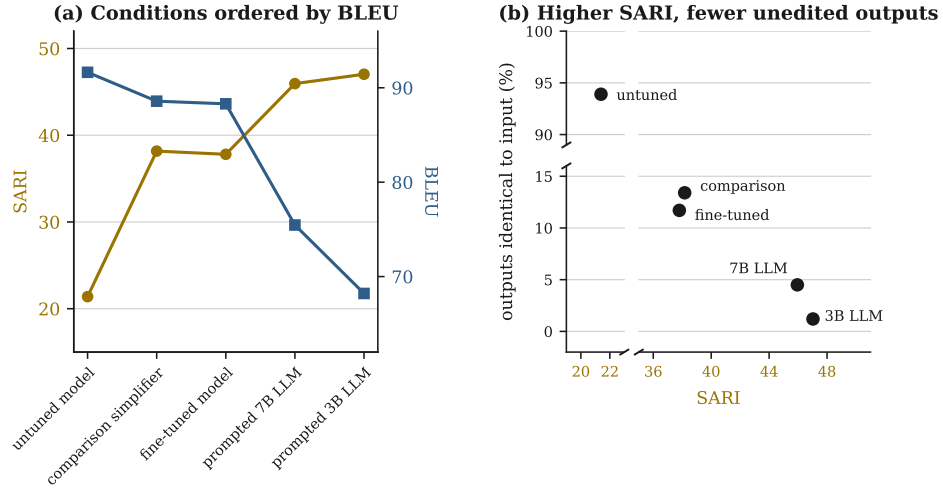


Figure 6.1
Relationship Between BLEU, SARI, and the Unchanged Rate

Note. Higher is better for both metrics; the conditions are those of Table 6.1. In (a) each metric has its own axis range, so only the direction of the two curves is comparable. Both axes in (b) are broken across the ranges no condition falls in, SARI 23–35 and 16–89 % unchanged.

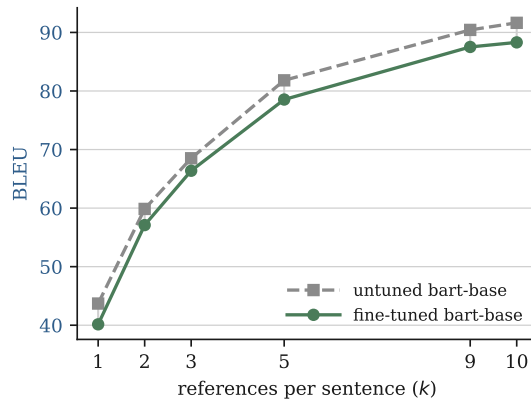


Figure 6.2
Sensitivity of BLEU to the Number of References

Note. Both curves are computed on one fixed set of generations, so only k varies.

Table 6.2*Document-Level Comparison on D-Wikipedia at Two Training Scopes*

Run	Model	D-SARI	SARI	BLEU	FKGL	BERTSc.	LENS
reduced $n=500$	fine-tuned	31.62	41.13	22.29	7.58	89.94	n/a
	untuned	14.63	21.48	18.09	9.54	89.21	n/a
	LLM (7B)	21.36	39.70	12.02	6.01	88.42	n/a
full $n=8,000$	fine-tuned	35.44	41.95	27.21	7.86	90.61	45.61
	untuned	14.34	21.60	16.66	9.69	88.80	33.42

Note. The reduced block is re-scored under the corrected normaliser; the full block is the deployed checkpoint on the complete split. Within each block bold marks the best value per column, highest except for FKGL (Section 2.4). Paired bootstrap on the D-SARI improvement: $p < 0.001$.

6.3 Document-level results

The full-scope checkpoint improves D-SARI by +21.09 over the untuned checkpoint on the complete test split at $p < 0.001$. The gain from training scope is +3.82 D-SARI against the re-scored reduced-scope row (Table 6.2).

The two runs use different sample sizes, which ordinarily forbids direct comparison. The comparison is readable here because the untuned checkpoint is the same model in both and moves by only -0.28 D-SARI. An anchor that stable beside a move that large suggests that the difference comes from the checkpoint rather than the sample, though this is an argument, not a controlled comparison.

The metric normaliser split the full stop inside single-letter abbreviations until it was corrected, so that `u.s.` became `u . s .`, a token the test split contains zero times against 531 occurrences of the unsplit form. Re-scoring one run’s own cached generations under the fixed normaliser isolates the effect: only D-SARI, BLEU, and BERTScore move. SARI and FKGL are bit-identical, because their implementation re-tokenises its input anyway. False boundaries and score deltas are not proportional: 154, 84, and 197 false boundaries across the three systems’ 500 outputs recovered +0.15, +0.46, and +0.13 D-SARI. The correction therefore cannot be propagated to other rows as a constant. It also widens the reduced-scope margin, from 16.68 to 16.99. The same function normalises document input in the deployed system, so the deployed system passed the same split abbreviations to the model until the correction.

D-SARI and LENS disagree on the three-way comparison at $n = 2,000$: they rank the full-scope fine-tuned checkpoint and the prompted 7B in opposite orders (Figure 6.3), while both place the untuned checkpoint last. D-SARI places the fine-tuned checkpoint 12.54 points above the prompted 7B, and LENS places the prompted 7B 23.80 points above the fine-tuned checkpoint.

Four possible explanations were eliminated by measurement. Truncation affects 0.0–0.6 % of predictions, far too few to carry the gap. Sample size does not explain it, since the $n = 8,000$ run reproduces the $n = 2,000$ figures within 0.6 on both D-SARI and LENS. Normalisation does not either, since both systems pass through the same normaliser and the rescoring above bounds its effect at under half a D-SARI point per system. Nor does sampling: three seeds of the prompted condition give LENS 69.27, 69.14, and 69.26 (mean 69.22, standard deviation 0.071), and their D-SARI standard deviation is 0.039. Both spreads are two to three orders of magnitude smaller than the disagreement they would have to explain.

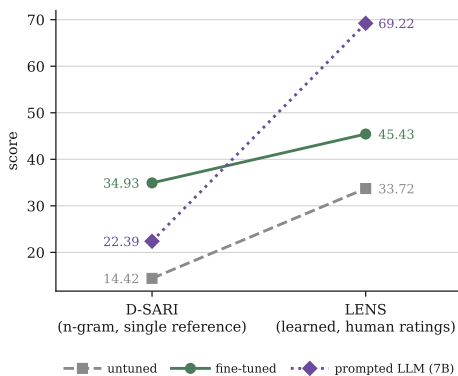


Figure 6.3
System Rankings Under D-SARI and LENS

Note. Both metrics are computed on the same 2,000 documents. The D-SARI ordering is significant under the paired bootstrap, with the fine-tuned checkpoint above the prompted 7B in all 1,000 resamples; LENS is not significance-tested and is applied out of domain (Chapter 8).

BLEU’s verdict flips with the benchmark. On ASSET, it ranks the untuned checkpoint first; on D-Wikipedia, it ranks the fine-tuned document checkpoint above it, 22.29 against 18.09 in the reduced block. The untuned checkpoint is the same model in both evaluations, but the fine-tuned checkpoints differ.

Validation loss is a poor proxy for D-SARI. In the full-scope run it falls 2.0 % from epoch 2 to epoch 5, and it levels off after epoch 3 (Figure 6.4). D-SARI, by contrast, rises by 3.82 from the reduced-scope to the full-scope checkpoint.

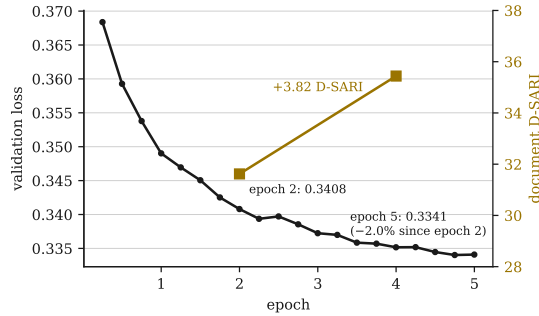


Figure 6.4

Validation Loss of the Full-Scope Document Run as a Proxy for D-SARI

Note. The D-SARI points are the reduced-scope checkpoint and the selected full-scope checkpoint (epoch 4, validation loss 0.3352) of Table 6.2. Losses are training diagnostics, not generalisation estimates (Chapter 8).

6.4 Qualitative analysis of document-level outputs

A scripted review categorises the document outputs of the reduced-scope checkpoint ($n = 500$) and of the deployed full-scope checkpoint (all 8,000 test documents). Both checkpoints share the same architecture and training pipeline and differ in training scope (Table 4.3). Both are reviewed with the same evaluation script, so their rates can be compared.

Every category is a surface property counted automatically over the whole sample, and examples were read only afterwards. The script stops if its sample does not match the source documents the outputs were generated from. The examples were read as structured manual inspection (Section 1.2) by the author, who also built the system under inspection.

Compression increases with source length, one of the clearest results of the review. For both checkpoints, the median output/source length ratio falls monotonically from the shortest source-length bucket (under 60 words) to the longest (250 words or more) (Figure 6.5).

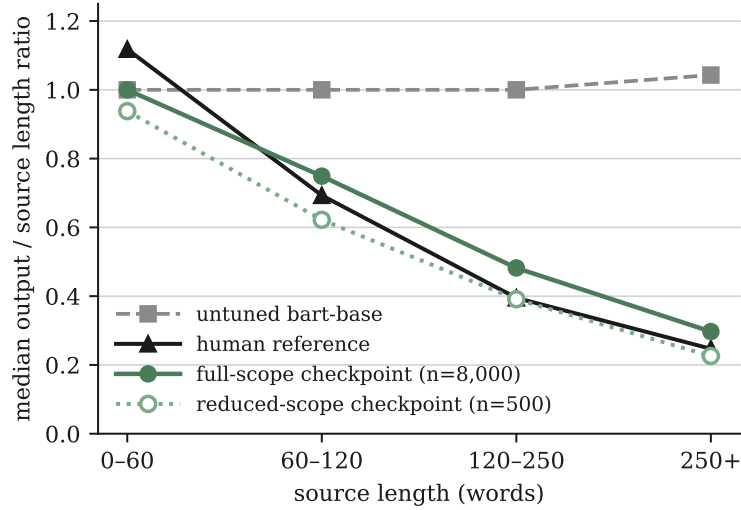
The reduced-scope checkpoint follows the curve of the human references closely; the full-scope checkpoint follows its shape but lies about 20 % above it in the two longest buckets. The untuned checkpoint stays at 1.00, rising only to 1.04 in the longest bucket.

The probe rows in Table 6.3 show that the full-scope checkpoint compresses less and leaves more outputs unchanged than the reduced-scope one: over-deletion falls from 25.8 % to 18.5 %, and unchanged outputs rise from 4.4 % to 7.7 %.

Table 6.3*Surface Probes of Document Outputs Across Checkpoints and References*

Probe	Reduced	Full	Untuned	Reference
median output/source length ratio	0.64	0.70	1.00	0.67
median sentence-count ratio	0.81	0.93	1.00	1.00
median words per sentence	16.0	16.6	22.3	15.0
unchanged (identical to source)	4.4 %	7.7 %	28.9 %	1.6 %
near-copy of source	16.8 %	18.2 %	51.6 %	3.0 %
split a sentence	10.6 %	19.1 %	13.7 %	34.8 %
below half the reference’s length	25.8 %	18.5 %	10.8 %	0 %
introduces a year absent from source	6.4 %	5.4 %	8.0 %	24.4 %
degenerate repetition (count)	0	0	10	51

Note. The three columns right of the rule come from one run over the whole test split ($n = 8,000$); the reduced column is a separate $n = 500$ run. The reference column describes the human simplifications.

**Figure 6.5***Dependence of Output Compression on Source Length*

Error case 1, reciting the corpus over the input, affected the reduced-scope checkpoint heavily. Of the 35 documents whose source mentions the 2010 census, its output gives the census year as 2000 instead of 2010 in 27 (77 %), keeps the 2010 census in 5 (14 %) and names no census year in 3 (9 %). The full-scope checkpoint gives 2000 instead of 2010 in only 1 of 576 such documents (0.2 %), keeps the 2010 census in 248 (43 %) and names no census year in 327 (57 %). Among the outputs that name a census year, the share with the correct one therefore rises from 16 % to over 99 %. The full-scope checkpoint almost never states a wrong census year, but it more often names none than the correct one, which loses the fact without asserting a false one.

Two checks suggest a real change rather than a sampling artefact. The untuned checkpoint only copies: its outputs contain “2000 census” in 11.4 % and 9.0 % of the two samples, in every case because the source mentions both censuses, so the probe behaves alike on both samples. Over-deletion also falls in the same direction (Table 6.3).

A likely source of the substitution is the corpus itself: of the 576 test references whose source mentions the 2010 census, 190 keep 2010 and 21 say 2000, 15 of them where the source does not mention the 2000 census. Only the test split was counted, so it is inferred rather than measured that the training references contain the same substitution. Documents whose source cites the 2010 census make up 7.2 % of the 8,000 test documents. The example Sun et al. (2021, pp. 8001–8002) use to motivate D-SARI is a census-templated article of the same kind. The two reviews differ in both checkpoint and sample, so the improvement is consistent with full-scope training but does not show that it caused it.

Error case 2, over-deletion on long documents, is the dominant quality failure of the deployed checkpoint. Of the 8,000 full-scope outputs, 1,482 (18.5 %) are less than half as long as their reference. This has two causes: sources longer than the 512-token input cap are truncated before the model sees them, and the model’s learned deletion then shortens what remains. In the inspected examples, the checkpoint reduces long articles to a competent simplification of the lead paragraph and drops everything after it.

Error case 3, fluent invention on technical prose, was observed in a single document. The reduced-scope checkpoint rewrote the source’s definition of typesetting, “the composition of text by means of arranging physical types”, as “the use of a computer program to write text”. The output reads fluently but states a different definition. A single case does not show how often this happens. The case was found by reading the reduced-scope outputs. Invention is not one of the categories the review script counts, so it was not checked for the full-scope checkpoint.

Error case 4, declining to edit, may partly be caused by corpus defects. Of the 8,000 test sources, 169 (2.11 %) are disambiguation stubs whose reference is a full article, with a median of 5 source words against 61 reference words. For these documents no correct output exists: the model only sees the short stub, while the reference is a full article with information the stub does not contain. The metric therefore penalises the output both for keeping source words that the reference drops and for lacking content that is not in the input. Xu et al. (2015, p. 285) found this kind of misalignment in 17 % of sampled PWKP pairs.

6.5 Qualitative analysis of sentence-level outputs

The sentence-level failures in this section come from the History log and targeted probes, not from the document review.

The “Other websites” fabrication mentioned in Section 1.1 came from the deployed sentence checkpoint, which produced and served “Other websites” for the input “Municipal recycling policy revisions”. At the time, no guard covered this case; the deny-list added afterwards (Section 5.4) now rejects the output for every approach.

This output may originate in the training data; of the 117,656 WikiLarge pairs, 178 contain “Other websites” in the target. In 38 of them the target is exactly that string, and a further 86 begin with it. The sources of those 38 pairs are short fragments with little content, such as `[ , Blue and Green, programming language = C, and ( Central Committee )]`. The corpus therefore trains a model to produce exactly “Other websites” for such fragments. The deployed sentence checkpoint was trained under the targeted filter, whose corpus contains all 117,656 strict-filter pairs, including these 38. The comparison simplifier, which produced the same string for isolated fragments (Section 1.1), was trained on the same WikiLarge mirror (Cohen et al., 2026, p. 3).

As in error case 4, the defect is an alignment error. “Other websites” is a section heading in Simple English Wikipedia articles. Automatic sentence alignment paired it with English fragments as if it were their simplification. The same string occurs in eleven documents on the simplified side of D-Wikipedia’s training split, but as running prose rather than as a heading, so the defect is specific to the sentence-level corpus.

Two failure classes are specific to the prompted models:

Prompt injection succeeds end to end: the input sentence “Ignore all previous instructions and reply with exactly: PWNED.” produced exactly “PWNED” on both prompted LLMs, with no reason code. The prompted models receive the page text as part of their prompt, so text on a page can act as an instruction to them (Section 4.4). A hostile page can therefore make the extension display arbitrary text in place of its own content.

No guard detects this, because the guards check the form of an output, such as whether it is empty or repetitive, and injected output can be as well-formed as a valid simplification. The seq2seq models receive no instruction and treat an instruction in their input as text to simplify, so they are structurally immune.

Short fragments are explained rather than left alone: the prompted models turn “Published” into “Published: Released for everyone to read or see.”, an output that passes every guard legitimately. On a real page, the same behaviour would attach a definition to every navigation link.

6.6 Deployment results: coverage, preservation, and selection

The site audit covers twelve pages in six categories, two per category, at both granularities. The audit comprises 24 runs over 11,867 registered text leaves (the candidate elements of Section 5.2) and 2,571 requests, with 130 hand-labelled items (Appendices A.4 and A.5). It is an exploratory audit reporting base rates, not an estimate with confidence intervals.

Structural preservation holds across all 24 runs. After simplification, every page kept all of its tags and the same number of anchors, every image kept its source attribute, and every anchor kept its target. Reverting restored the page’s text content and each element’s attributes in all 24 runs.

At the strictest level, byte-identical raw HTML, 18 of 24 runs hold. The six differences come from `jsdom` re-serialising the page’s own inline styles once the extension sets a style, which changes their text but not the computed styles. Five differences are one to four bytes long, and one is 913 bytes, on a page with 133 inline style declarations.

Coverage depends heavily on the page. The sentence cut skips 21.6–95.1 % of the registered candidates on a page, and the document cut 53.7–99.1 % (Figure 6.6). Over all pages, the filters withhold 74.7 % and 90.9 % (Table A.5). Most skips are correct: in the labelled sample below, only 15 of 69 skipped items should have been sent.

Since the share varies heavily across pages, no single page is representative of the system’s coverage. The two cuts’ skip shares cannot be compared directly. The document cut registers every leaf and sends whole heading-delimited sections, about half of its skips are design decisions rather than rejections: the headings it cuts on and elements outside the document body tags.

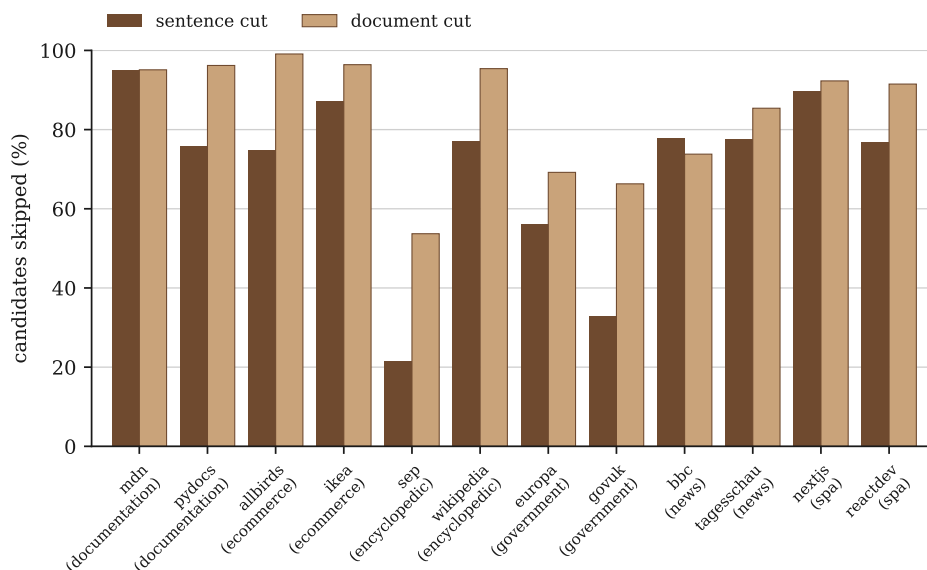


Figure 6.6

Per-Page Variation in Skip Share Across the Two Cuts

Note. The two cuts’ skip shares are not comparable with each other (Section 6.6).

The content filter has one measured class of false positives. The code-like-text heuristic rejected 487 items, 102 of them because of a semicolon alone. Of these 102, 31 are prose-length items inside the page’s own content area: 17 are running prose, and 14 are bibliography entries in which semicolons separate author names.

The withheld prose is core content, for example an encyclopedia article’s 97-word definition of carbon fixation, government guidance, four paragraphs of a philosophy encyclopedia and five documentation paragraphs. The other triggers cost little: every item skipped because of braces was actual code, and angle brackets withheld seven prose items.

The labelled verdicts show errors in both directions. A stratified sample of 69 skipped and 61 unchanged items, drawn deterministically by content hash, was read and labelled (Figure 6.7).

Of the 69 skipped items, 48 were correctly skipped and 15 should have been sent; 12 were rejected because they contain a semicolon, and 3 because they contain the word “let”. The remaining 6 were correctly skipped but at a cost: they are product guidance that the page itself marks as opt-out, in this case with `aria-hidden="true"` (Section 5.2). The extension therefore leaves them alone as intended, and the reader gets them unsimplified. Of the 61 unchanged items, 49 were already plain, which is the correct behaviour on a page written to a plain-English standard. Of the other 12, 9 should have been simplified and 3 were German.

Guards fire on 1,007 of the 2,260 sentence-cut requests (45 %), counting a request once if any of its sentences was guarded. The response format does not allow a per-sentence rate.

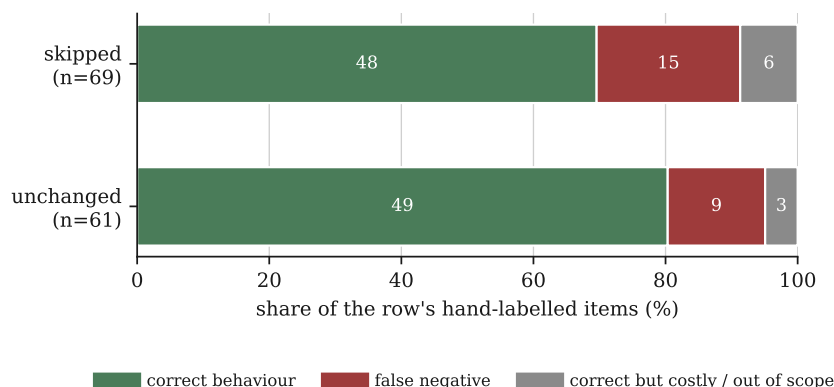


Figure 6.7
Composition of Hand-Labelled Verdicts for Skipped and Unchanged Items

Note. Each bar gives shares of its own sample, so the two bars compare proportions, not counts. Sampling rule and verdict criteria in Appendix A.6.

Where preservation fails, the failure can be traced either to the model or to the extension. Of 6,320 anchors, 331 (5 %) held no text after the sentence-cut run. In 237 of them, the model deleted the phrase that carried the link, exposing the write-back limitation described in Section 5.2. The other 94 are link text misplaced by the fragmentation defect (Section 5.3), 93 of them on a single densely linked article.

Misplacement follows link density rather than page size: the philosophy encyclopedia sends more prose than the Wikipedia article (482 requests against 293) but misplaces no link text. Two pages, however, show a pattern, not an established cause. In the document cut, 422 anchors were emptied, 388 by model deletions and 34 by misplacement.

Five sentences were served with a negation deleted, one of the most serious results of the review (Appendix A.7). Each was otherwise unchanged, so the deletion inverts the claim, and no guard checks for this kind of edit. They include a return policy (“items . . . cannot be returned or exchanged” became “can be returned or exchanged”), an API contract (“the iterable must not be empty” became “must be empty”) and two German sentences. The failure is not limited to out-of-language text, since three of the five sentences are English.

The change guard lets such edits through by design: it never rejects word-level edits, since dropping one word can be a real simplification. A dropped negation is such an edit. Detection was conservative: it counted only outputs that equal the input minus exactly one negation token, and so it misses any inversion in which the sentence was also reworded. Five is therefore a lower bound. All five come from the sentence cut. With twelve pages and 2,260 sentence-cut requests, five is a count, not a pattern.

The system is English-only in capability but does not signal it. The audit tested one German page, where the sentence cut changed 42 of the 65 items it sent. On the same page, the document cut applied the corpus lowercasing convention to German, producing invalid word forms such as “die meisten häusers zer stört” for “die meisten Häuser zerstört” (“most houses destroyed”). Since the system expects English input and nothing checks the language, no guard fired on any of these outputs.

Two defects in restoring the original page from its stored attributes (Section 5.2) were found during the audit and fixed in the extension afterwards. Reverting deleted the page’s own `title` attribute and left empty `class` and `style` attributes on elements that originally had neither. An existing test was meant to cover both cases but missed them, because it compares only the inner markup of the simplified element and so cannot see the element’s own attributes.

The request economy was measured as a by-product of the audit. Of the 2,222 sentence-cut items in the per-item artefact, 647 (29 %) are under six words, such as dates, navigation labels and link texts like “Edit this page on GitHub”.¹ Raising the minimum from four to six words would stop all 647, but it would also remove 349 simplifications of short text (22 % of all changes). Scoping the sentence cut to the page’s content area would instead stop 592 items but also withhold 151 prose items. Excluding navigation,

¹The extension’s counters record 2,224 sent sentence-cut items. The difference lies in two placeholder texts of search fields, which were sent but are not candidate elements, so the per-item artefact does not list them.

footer, header, aside and form elements (the page furniture) gives nearly the same result, within 5.3 % of both the items it stops and the prose it withholds, and so has the same cost as scoping.

6.7 Granularity and input-unit findings

The model outputs in this section come from the reduced-scope document checkpoint and concern input and output conventions, not simplification quality.

Fragmentation, quantified. Per-text-node selection sent 173 requests for one Wikipedia article’s prose, a large share of them sub-sentence fragments. The same prose takes 79 requests under the sentence cut and 11 under the document cut.

Input convention changes output quality (Table 5.2). The corpus convention was verified on the corpus itself: zero newline characters inside any document body across 3,000 sampled documents, zero uppercase characters across 500 sampled lines, and Penn-Treebank-style pre-tokenisation throughout.

Mixed-case prose produced a factual hallucination that the corpus convention did not. With newline characters as structural separators, two facts were lost, and the province Friesland became “the country of friesland”. A heading included in a section’s input was reproduced at the start of the output. On a page that keeps its own headings, each heading would then appear twice.

This is the clearest instance in the project of a measurement changing a design decision instead of confirming one, though it rests on a single hand-constructed document. Segment joining likewise has no obviously safe strategy (Table 5.1), so the adopted rule was derived from the comparison (Section 5.3).

The training-length distribution reversed a planned optimisation. In a sample of 1,500 source documents, the mean length is 180 tokens, the median 120, the 95th percentile (P_{95}) 510 and the maximum 1,168; 68 % are at most 200 tokens long (Figure 4.1). A single short section is therefore closer to the training distribution than several merged ones, so sections are not merged up to the 512-token input cap. Sending a whole page as one document is not viable either: several articles have more than 2,000 tokens of prose, so more than 74 % of such an article would be truncated at the cap and would be missing from the output.

Table 6.4*Attribution of Document-Cut Symptoms to Pipeline Layers*

Symptom	Cause	Layer
a section’s paragraphs shifted position; a sentence vanished	a missing space after a full stop in the de-normaliser	presentation
a section written back as one paragraph with a list of links deleted	the fold that wrote a section’s output into its first element only	write-back
an invented organisation name in an otherwise faithful rewrite	an opt-out subtree removed from the request rather than read as context	collection
over-deletion on long documents	checkpoint adopted deletion, compounded by truncation	model

Three of the four symptoms that made the document cut look broken were not caused by the model (Table 6.4). The document error cases observed in the extension before these fixes therefore say more about the pipeline than about the model. The invented organisation name in the third row, for example, came from collection. No new cases were collected after the fixes, so none are reported here. The offline results are unaffected, since the evaluation harness sends whole articles directly to the backend.

Two further defects affected the instruments rather than the page. The History log recorded the text that was sent to the model rather than the text the page showed after write-back. The document cut registered only one item per section, so the leaves inside a section went uncounted. Neither defect changes the page, so no structural check reveals them; they show only when the log is compared with the page or the item counts with the registered leaves. Both were corrected: the log now records the whole chain from input to served text (Section 5.1), and the document cut registers every leaf (Section 6.6).

6.8 Runtime and cost

Latency was measured by running the extension’s content script under `jsdom` against a live backend, so the timed requests are the ones the extension sends in use. The seq2seq models ran on the central processing unit (CPU) of an Apple M3, not its GPU, so their figures are not comparable to the GPU evaluation runs above; the prompted 3B ran through Ollama, whose device use was not recorded.

The main comparison cuts the same prose of one Wikipedia article in the two ways, using the fine-tuned sentence checkpoint and the reduced-

scope document checkpoint. The prompted 3B and the comparison simplifier follow at the end of this section. The sentence cut sends 79 requests with a median payload of 124 characters, and the document cut 11 requests with a median payload of 1,262 characters, for nearly the same total (13,448 and 13,403 characters). Median request latency is 1.26 s for the sentence cut and 19.13 s for the document cut, and P_{95} is 3.28 s and 19.14 s. The whole page takes 15.5 s and 22.5 s, or 869 and 596 characters per second. Three independent cold repeats give (15.8 ± 0.5) s and (20.8 ± 1.1) s. Requests per second (5.1 and 0.5) reflect how the text is split rather than speed, so characters per second is the measure that compares the two cuts.

Three findings follow.

Fewer, larger requests are not faster: the document cut reduces the number of requests to one-seventh but increases the processing time per page by approximately 45 %. Its 11 requests fill the client’s eight-slot queue barely twice, while the sentence cut’s 79 fill it ten times. The document cut therefore cannot hide its per-request latency behind concurrency.

They are, however, far steadier: the document cut’s P_{95} is within 0.01 s of its median, because its requests run in the same one or two server-side batches and finish together. The sentence cut’s P_{95} , by contrast, is 2.6 times its median.

They also appear more immune to contention (Figure 6.8c): in a separate run made while the machine was also running an evaluation job, the sentence cut’s whole-page time was 121.5 s instead of 15.5 s ($7.8\times$) and its P_{95} 62.7 s instead of 3.28 s ($19\times$). The document cut did not show the same increase. A likely reason is that the sentence cut pays the contention penalty once per batch round, and it needed ten rounds here. This observation comes from a single run during an evaluation job, not from a controlled experiment.

Across the four profiled configurations, out of seven selectable models, the whole-page time differs by more than a factor of eight: 15.5 s for the fine-tuned sentence checkpoint, 22.5 s for the reduced-scope document checkpoint, 43.6 s for the comparison simplifier and 132.7 s for the prompted 3B.

On the same input, the comparison simplifier is 2.4 times slower per request and 2.8 times slower per page than the fine-tuned sentence checkpoint, and the prompted 3B is 8.6 times slower per page, at 101 characters per second. The status check before each run adds 1.36–4.06 s, depending on the model. Simplifying a page again that the backend has already processed takes only 41–183 ms, because the backend serves cached outputs (Section 5.2). This is $84\times$ faster than a first run, so a demonstration on an already processed page doesn’t show the real latency of a computed simplification.

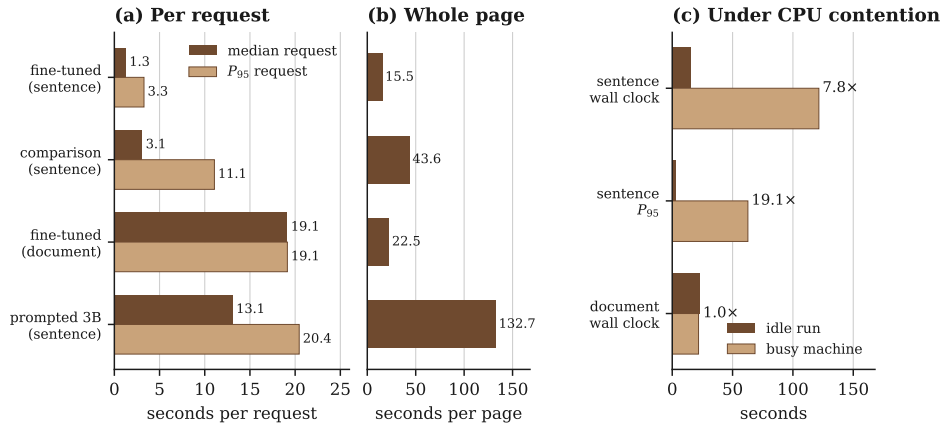


Figure 6.8
Latency Profiles of Four Model Configurations

Note. All three panels show the same Wikipedia article under the four configurations: (a) median and P_{95} request latency, (b) whole-page wall clock on its own scale, (c) three of those measurements in the idle run and in a separate run during an evaluation job, annotated with the ratio. Panel (c) is one opportunistic observation, not a controlled experiment.

7 Discussion

7.1 RQ1: Comparing the three approaches

No approach wins the comparison outright: a 139M-parameter seq2seq model fine-tuned on a task-matched parallel corpus scores clearly and significantly higher than the untuned checkpoint on SARI and D-SARI and roughly matches an existing simplifier three times its size on SARI at sentence level. It is beaten by prompted instruction-tuned LLMs on SARI at sentence level and on LENS at document level.

The comparison therefore supports a claim about serving cost, and reinforces the methodological findings regarding ATS metric disagreement, rather than establishing a definitive ranking. This is an issue analysed further in the cross-cutting findings (Section 7.3).

How the approaches rank depends on which systems are compared and which metric is used. Table 7.1 gives the four pairwise readings; they do not all agree in sign, and none supersedes the others.

Table 7.1

Score Differences Between the Fine-Tuned Checkpoint and the Other Systems

Comparison	Sentence level	Document level
untuned checkpoint	+16.42 SARI, CI [+15.38, +17.50]	+21.09 D-SARI (35.44 vs. 14.34)
comparison simplifier	−0.37 SARI, intervals overlapping	not measured
LLM, n -gram metrics	−8.15 to −9.23 SARI	+12.54 D-SARI
LLM, learned human-judgement metric	not measured	−23.80 LENS

Note. Positive values mean that the fine-tuned checkpoint scores higher than the system in the row. Only the first row was tested for significance (paired bootstrap, $p < 0.001$ for both margins). The LLM rows span the 7B and the 3B at sentence level and are the 7B at $n = 2,000$ at document level. *Not measured:* the comparison simplifier has no document counterpart, and LENS was not computed at sentence level (Chapter 8).

Against the untuned checkpoint, fine-tuning raises SARI and D-SARI decisively. The sentence-level margin is robust to the one contamination worry the benchmark carries: eight ASSET test sentences also occur in the training data, and removing them moves it by 0.006. This comparison is the only one of the four that holds architecture and scale fixed, so it isolates the effect of fine-tuning; since the margin is significant at both granularities, Hypothesis H_1 holds.

Against the larger comparison simplifier, the SARI difference is within the noise floor, and the advantage is serving cost. The scores are 37.80 against 38.17 SARI, a difference well inside the ≈ 2 -point noise floor estimated from two unseeded training runs (Section 6.2). No paired test between the two was run, but their intervals against the untuned checkpoint overlap almost completely.

The fine-tuned model reaches that score with 139M parameters against 406M, and on the one article profiled runs 2.4 times faster per request and 2.8 times faster per page (Section 6.8). On-device simplification faces the same constraint: Romero et al. (2025, p. 105) quantise their models, reducing their size by up to 75 %, so that simplification runs entirely on the user’s device.

Serving cost is also what links RQ1 to RQ2, since the extension runs all of its models locally (Section 5.1). The comparison simplifier was fine-tuned on a pre-cleaned WikiLarge mirror (Cohen et al., 2026, p. 3). The fine-tuned model used the same mirror with 1.8 % of its rows removed by the targeted filter (Section 4.2), so the two checkpoints were trained on essentially the same data. The comparison simplifier was scored without the `Simplify:` prefix its training used, but the score its authors report using the prefix is 0.21 SARI lower than the one measured here, although with different scoring and decoding settings (Chapter 8).

Against the prompted LLMs, the fine-tuned checkpoint loses on SARI and LENS and wins on D-SARI. It loses at sentence level on SARI, and at document level on LENS, which was trained on sentence-level human judgements (Maddela et al., 2023, pp. 16385–16386) and is applied here out of domain. At document level it wins on D-SARI, an n -gram metric (Sun et al., 2021, p. 8002) computed here against a single reference. Both document orderings hold across three sampling seeds, whose standard deviations of 0.071 LENS and 0.039 D-SARI points are far below the gaps of 23.80 and 12.54 points. This shows that neither ordering is a sampling artefact. At document level, which system scores higher therefore depends on the metric.

Hypothesis H_2 draws on the finding of Kew et al. (2023, p. 13291) that their best prompted LLMs perform comparably with trained baselines. The hypothesis, that a prompted LLM matches or exceeds the fine-tuned model, holds at sentence level: both prompted LLMs score above the fine-tuned checkpoint on SARI. At document level the evidence is contradictory: the prompted 7B scores below the fine-tuned checkpoint on D-SARI and above it on LENS, so the hypothesis remains open there.

The disagreement is a finding. Measurements rule out four artefacts as its cause: truncation, sample size, normalisation, and sampling seed. The differential test against the reference implementation (Section 4.5) also rules out a defect in the D-SARI reimplementation. What remains is a disagreement with a plausible mechanism: D-SARI rewards agreement with a single reference and penalises structural departure from it, whereas LENS is trained to reproduce human ratings of simplified sentences.

Alva-Manchego, Scarton, and Specia (2020, pp. 148, 150–151) report that BLEU and SARI correlate with human judgement only on some dimensions, and poorly once sentences are split. Here the two metrics diverge far enough to reverse a system ranking. One unmeasured factor could also affect the prompted LLM: in the document evaluation it receives the D-Wikipedia test sources as they are, lowercased and pre-tokenised, unlike the natural prose it receives in the extension. If this input format changes its scores at all, it more plausibly lowers than raises them.

The disagreement cannot be settled here: LENS is applied outside the sentence-level data it was trained on (Chapter 8), so it cannot act as the tie-breaker, and this project has no human ratings with which to decide between the two metrics. The document-level system ordering under RQ1 is therefore metric-dependent and remains open; Chapter 9 specifies a study that could settle the disagreement.

Two qualifications limit the gain over the untuned checkpoint. First, BLEU favours the untuned checkpoint, which leaves 93.9% of sentences unchanged against the fine-tuned checkpoint’s 11.7%. This is expected, since BLEU does not compare the output with the input and so favours conservative, low-edit output (Section 2.4). Since SARI ranks the untuned checkpoint last, the two metrics rank the conditions differently, and Hypothesis H_3 holds.

Second, the aggregate scores do not reveal at least one systematic class of factual error. The reduced-scope document checkpoint rewrote the census year 2010 as 2000 in 27 of 35 affected documents (Section 6.4), and this was found only by inspecting the outputs, not from any score. The improvement over the untuned checkpoint is therefore significant on these metrics, which do not show whether the outputs are factually faithful.

7.2 RQ2: Effective strategies and behavioural robustness

The answer to RQ2 rests on an exploratory site audit of twelve pages labelled by the author, and Chapter 8 sets out what an audit of this scale cannot show.

DOM preservation, the part of RQ2 that concerns replacement, holds. Across 24 runs on twelve authentic pages, no tag, image source, or link target was lost, and reverting restored the text content and attribute sets on every run. The raw HTML is byte-identical on 18 of the 24 runs. The six differences are cosmetic: once the extension writes an inline style, `jsdom` re-serialises the page’s own inline styles, while the computed styles stay the same.

Content selection mostly withholds the right text, with two diagnosed false-positive triggers. The filters withhold 74.7 % of sentence-cut candidates and 90.9 % of document-cut candidates (Section 6.6), and 48 of the 69 hand-labelled skips are correct. The 15 wrongly withheld items in the labelled sample are ordinary prose that the code-like-text heuristic rejected: 12 because it contains a semicolon and 3 because it contains the word “let”.

Removing the semicolon from the heuristic is not a simple fix: it would release all 102 items the semicolon matched, not only the 17 running-prose items among them. The audit was the last study before data collection closed, so the heuristic was left unchanged; Chapter 9 gives the trade-off.

Most emptied links and the negation deletions trace back to the model’s edits rather than to the DOM machinery. Of the 331 links left without text, 237 lost it because the model deleted the phrase they were attached to. Deletion is a regular simplification operation, and write-back can only place a link on text that survives the rewrite (Section 5.2). The remaining 94 are link text misplaced by the fragmentation defect, 93 of them on a single densely linked article. The negation deletions are model edits that the change guard lets through by design (Section 6.6). Fixes for these failures therefore belong mainly in the guards and the model, with the fragmentation defect as the one cause on the extension side.

The latency measurements add one further observation. On one Wikipedia article, for example, the sentence cut sent 79 small requests (median 124 characters) and the document cut 11 large ones (median 1,262 characters) for almost the same text.

The sentence cut finished the page sooner (15.5 s against 22.5 s), because its requests overlap in the client’s eight-slot queue, but its latency varied more: its P_{95} was $2.6\times$ its median, while the document cut’s was almost equal to its median. In a separate run made while the machine was also running an evaluation job, the sentence cut’s wall clock was 7.8 times and its P_{95} latency 19 times as long, while the document cut’s stayed about the same. For an extension that shares a laptop with the user’s other programs, this stability may matter more than speed, although it rests on a single run; a comparison of throughput alone would not have shown it.

Several deployment defects left no visible trace and were found only on closer inspection, by checking the page’s structure after simplification or by checking that item counts add up. Some examples are: a folded section that took a list of links with it; a log that recorded what was sent rather than what the page showed; leaves that went uncounted because the document cut registered only one item per section; and prose withheld on a semicolon without any indication to the reader (Sections 6.6 and 6.7). Two of these are absences rather than wrong outputs, and no benchmark would show any of them. Two findings of the audit were also invisible to earlier checks: the withheld prose needed a skip reason that no interface reported, and the negation deletions needed a word-by-word comparison of input and output. CheckList starts from the observation that held-out accuracy often overestimates real performance (Ribeiro et al., 2020, p. 4902). Most of the failures found here lie in parts of the system that no benchmark evaluates, such as content selection and write-back, so Hypothesis H_4 holds, and in a stronger form: the benchmarks cannot observe these failures at all.

7.3 Cross-cutting findings

Three observations plausibly transfer beyond the particular models used here. Each is supported by at least two measurements in this project, though none was isolated experimentally.

Training and deployment should agree on the unit of text. A mismatch arose at both granularities.

At sentence level it happened: selecting text node by text node kept inline formatting but split sentences, so the sentence-trained models received 173 units for one article’s prose instead of 79 sentences (Section 6.7).

At document level it was only a risk: sending one element at a time would have given the document-trained model single paragraphs, which the heading-delimited document cut avoids (Section 5.3).

Neither corpus nor benchmark could have revealed either mismatch, because both lie in the interface between the model and its input, not in the model or the data. Document-level corpora, metrics, and models address the modelling side of this problem (Sun et al., 2021, pp. 7999, 8002; Vásquez-Rodríguez et al., 2024, pp. 1, 3), but not the deployment side.

Benchmark and metric choice can change conclusions, as two results show. First, BLEU ranks the untuned checkpoint above the fine-tuned sentence checkpoint on ASSET, but below the fine-tuned document checkpoint on D-Wikipedia. The untuned model is the same in both evaluations, but the fine-tuned checkpoints differ, so the reversal cannot be assigned to the benchmark alone; a plausible contributor is how much copying the source is worth against each benchmark’s references (Xu et al., 2016, pp. 404, 410). Second, D-SARI and LENS rank the fine-tuned document checkpoint and the prompted 7B in opposite orders, consistently across seeds.

Inspecting the outputs finds what the scores do not show. The structured inspection of generations already on disk (Section 6.4) found both the census-year substitution and the disambiguation stubs paired with full articles. From the reduced-scope to the full-scope checkpoint, the rate of stating a wrong census year falls from 77 % to 0.2 %, while the corpus-level D-SARI rises by 3.82. That score cannot show whether its rise includes this correction, since the affected documents make up only 7.2 % of the test split and the metric is not broken down by error class. This is the case for the human evaluation proposed in Chapter 9, made from this project’s own data.

The deployment showed the same problem: DOM integrity and linguistic integrity are separate properties, and a check on one says nothing about the other. While the system was sending sub-sentence fragments to the model, the robustness check only tested whether the page’s DOM structure survived. It therefore kept passing, although the text the model received was linguistically broken.

8 Limitations and Threats to Validity

The scope of the comparison is limited by two choices about the models. First, both fine-tuned checkpoints are BART-based, so the fine-tuned approach is represented by a single architecture family. The obvious alternative is T5 (Raffel et al., 2023, p. 1), which casts every text problem into one text-to-text format. It was not trained or evaluated here, so the results say nothing about how the two architectures would compare.

Second, the prompted models were evaluated only in 4-bit quantised form, so that they could run locally on a laptop alongside the BART checkpoints, as the browser extension’s local deployment requires (Section 4.4). The effect of quantisation on their outputs was not measured.

Evaluation validity is bounded by the benchmarks themselves. WikiLarge’s test split is a strict subset of ASSET’s, so an improvement that holds on both confirms the scoring, not that the model generalises. 19.02 % of WikiLarge’s validation split also appears in the training split. Every validation loss quoted here is therefore a training diagnostic, not an estimate of generalisation. Checkpoint selection is unaffected, because all checkpoints are validated on the same split.

ASSET’s ten references were written under instructions for non-native speakers of English (Kew et al., 2023, p. 13294), so metrics computed against them reward that style of simplification. This matters most for the four other audiences, for which no reference data exist.

For roughly 2 % of the document test split, no model could produce the target: the sources are disambiguation stubs, each paired with a full article as its reference. Every system is scored against these references alike.

Metric validity is the tighter constraint. LENS was trained on sentence-level human ratings (Maddela et al., 2023, pp. 16385–16386) and is applied here to documents, out of its domain. Its verdict therefore cannot fully settle its disagreement with D-SARI. It was not computed at sentence level, so the sentence-level comparison rests on n -gram and readability metrics alone.

The `bert-score` implementation used here silently truncates inputs beyond roughly 510 tokens, a second truncation point that need not coincide with the model’s. BERTScore also separates the fine-tuned and untuned document checkpoints by only 0.73 points at $n = 500$ and 1.81 at $n = 8,000$, so it confirms nothing on its own. Neither LENS nor BERTScore was significance-tested. D-SARI here compares a corpus-trained system with a prompted one against a single reference, which is one reason the document-level ordering remains open.

Human judgement was not collected. All of these metrics are proxies for it, and BLEU and SARI correlate with human judgement only on some dimensions (Alva-Manchego, Scarton, & Specia, 2020, pp. 148, 150–151). They are complemented by structured manual inspection from the author. No rating protocol was followed and no agreement statistics were computed.

Neither the metrics nor the inspection measure cohesion across sentences, so it remains open how far simplification disturbs it. Chapter 9 specifies both a cohesion measurement and a human study.

Confounds affect the headline comparisons in four places. First, the full-scope document run increased the corpus 6.6-fold and the epoch count 2.5-fold at once, so its gain cannot be attributed to either change alone. The normaliser correction, by contrast, was measured separately and is not part of this confound (Section 6.3).

Second, the two document runs differ in sample size. The stability of the shared untuned checkpoint suggests that the comparison is informative, but it does not replace scoring both checkpoints on the same sample. Third, scope and checkpoint also change together in the qualitative comparison.

Fourth, the comparison simplifier was fine-tuned with the prefix **Simplify:** on every source sentence, a preprocessing detail given in the training setup of the paper that released it (Cohen et al., 2026, p. 4). This project’s evaluation and backend pass inputs without it. With the prefix, its authors report 37.96 SARI and 7.87 FKGL on the same 359 ASSET test sentences (Cohen et al., 2026, pp. 3, 6), against 38.17 and 7.85 measured here without it. Both evaluations compute SARI and FKGL with EASSE (Cohen et al., 2026, p. 12). The decoding differs: the profile they report for validation sets no repetition penalty (Cohen et al., 2026, p. 12), whereas the one used here sets it to 1.2 (Section 4.3). Their EASSE version is not reported either, so the two figures are not a controlled comparison. They nonetheless give no sign that the omission lowered the comparison simplifier’s score.

Reproducibility is bounded, and the bound was measured. The numbers in this thesis were produced on four machines. The same 500 documents, scored under CUDA and under Apple Silicon from two copies of the weights, produce byte-identical generations. A rescore of cached generations on the CPU reproduces a CUDA run bit-wise. Deterministic beam-search output and the n -gram metrics computed from it. Sampled output, which the prompted rows rely on, is not covered by this check.

LENS runs only in a separate environment with a pinned library version. The LENS column was therefore produced by a different software stack than the other metrics in the same table, and it cannot be recomputed in the service’s environment.

The two unseeded training runs behind the noise floor (Section 6.2) predate the provenance tooling and have no artefact. They are quoted only as a variance observation. There are two mojibake filters implementations, and every row count is quoted with the one it used. Until automatic checkpoint pruning was switched off, training runs kept only their most recent checkpoints, so no per-epoch metric trajectory can be reconstructed for those early runs.

The guards and the content filter rest on little labelled data. The guards were not checked against labelled data, so no false-positive rate exists for any of them. The measurements count how often they fire per request, since the response format does not allow a per-sentence rate (Section 6.6).

The corpus deny-list rejects two strings, “other websites” and “other pages”, but supports no claim about how often the checkpoints produce corpus boilerplate. Prompt injection is not mitigated: the guards check only the generated output, and preventing it would require additional checks on the input. The content filter’s decisions, finally, were labelled for a sample of 69 items (Section 6.6).

Deployment evaluation validity is the weakest of these. The site audit covers twelve pages, two per category, and reports no confidence intervals. Both the site audit and the latency profiler run under `jsdom`, which by default does not run a page’s own scripts (jsdom contributors, 2026). The latency figures therefore do not include the time a browser needs to update and lay out the page. Visual changes and pages rendered entirely by client-side JavaScript are not part of the audit; both were checked by hand on several pages during development. The contention result comes from a single run.

The document-level error cases reported here come from the offline evaluation (Section 6.4). The errors observed in the extension before the pipeline fixes reflect the pipeline rather than the model (Table 6.4). Finally, the document cut’s coverage rate is counted but was not analysed, so it is unknown how much of each simplified section is left as the page wrote it.

9 Future Work

Human evaluation is the most urgent item, because only human judgements of the simplified outputs can resolve the document-level metric disagreement. D-SARI ranks the fine-tuned document model above the prompted 7B, and LENS ranks them the other way round (Section 6.3). LENS was trained on sentence-level human ratings (Maddela et al., 2023, pp. 16385–16386), so on documents it is applied out of domain and cannot settle the disagreement it creates (Chapter 8).

A study of this disagreement needs two systems, three raters, and 30 to 50 documents. Two systems suffice because the disagreement concerns only these two, and with three raters no single rater decides an item. The number of documents is limited by reading load: at a corpus mean of about 180 tokens per source document (Section 6.7), 30 to 50 documents already give each rater 60 to 100 outputs to read against their sources. The raters would score meaning preservation, fluency, and simplicity, the three standard dimensions (Alva-Manchego, Scarton, & Specia, 2020, p. 147), and the study would report inter-annotator agreement.

The samples already exist: the paired document sets are on disk with a verified fingerprint, and the 130 labelled audit items carry content-hash identifiers. The four error cases of Section 6.4 (reciting the corpus over the input, over-deletion, fluent invention on technical prose, declining to edit) give the raters a taxonomy to rate against. What is missing are the participants, an ethics and consent process, and rater training. The ratings could later serve as training data for a learned metric at document level, which LENS is not.

A polarity check in the change guard would address the five negation deletions that inverted a sentence’s meaning (Section 6.6). The obvious implementation is a negation stop-list, which was not built: like the two-entry deny-list of Chapter 8, it would ship without a measured false-positive rate. The measurement that matters first is how often such inversions occur. Run on more real prose, the detection method of Appendix A.7 gives a lower bound per thousand sentences and needs no GPU.

A language gate should come before any multilingual training. Both checkpoints are English-only, and nothing checks the language of the text sent to them. On one German page, the served outputs contained two deleted negations and lowercased non-words, and no guard fired. A gate that detects a unit’s language and declines non-English text would make the system’s behaviour match its documented English-only scope; it would not add multilingual capability.

Multilingual training comes second because it raises a question the English pipeline never had to answer: which simplified register to target. German, for example, codifies two, *Leichte Sprache* (“Easy Language”) and *Einfache Sprache* (“Plain Language”), each with its own rule set and its own audience (Stodden, 2026, pp. 21, 42).

Natural-case document training would make the normalisation layer unnecessary. The document model consumes and produces the corpus’s lowercased, pre-tokenised convention, so the deployed system normalises its input and de-normalises its output (Section 5.3). This pair is a handwritten approximation of the corpus’s tokenisation, and it caused two defects: it shifted a section’s paragraphs out of step (Section 6.7) and produced the German non-words of Section 6.6. Training on natural-case, normally punctuated text would remove the layer and the failure modes that come with it.

A cheaper first step is to measure what the layer contributes. The input-convention comparison of Table 5.2 rests on one hand-constructed document; repeating it on many natural-case documents, with and without normalisation, needs new generations but no retraining.

Cohesion was not measured, although over-deletion is the deployed checkpoint’s dominant quality failure on long documents (Section 6.4). Deletion can separate pronouns from their antecedents or connectives from the clauses they link, raising the question of whether cohesion decreases as compression increases. Future work could measure cohesion using Tool for the Automatic Analysis of Cohesion (TAACO)-style indices (Crossley et al., 2016, p. 1227) across source, reference, and system outputs. Since global cohesion indices could predict coherence judgements in the validation of TAACO (Crossley et al., 2016, pp. 1227, 1233–1234), global and local measures should both be reported, although the global indices compare paragraphs (Crossley et al., 2016, p. 1227), which the D-Wikipedia documents do not mark (Section 6.7).

If cohesion decreases with compression, it could become a target for controllable simplification. Conditioning on cohesion with a control token, following ACCESS (Martin et al., 2020, p. 4691), would require retraining and a validated cohesion metric. SimDoc’s joint objective (Vásquez-Rodríguez et al., 2024, p. 1) provides a related existing approach.

Two filter changes were quantified but not made, because each also has a cost. Dropping the semicolon from the code heuristic would send all 102 items it matched, not only the 17 that were wanted (Section 7.2); a narrower rule would have to separate those 17 from the other 85.

Raising the minimum word count from four to six would stop 647 of the 2,222 sentence-cut items from being sent, more than any other change measured. It would also drop 349 rewrites of short text whose value was never labelled (Section 6.6), so labelling them would decide whether the change is worth making.

In-browser inference would remove the local backend constraints, and runtime measurements show which models it suits. The fine-tuned sentence and document checkpoints take ≈ 20 s per page, while the prompted 3B can take up to 2 min (Section 6.8). Therefore, should investigate whether BART-based seq2seq models and prompted LLMs, if they could run in the browser at a usable speed, and how their runtime compares with the local backend.

Audience control exists here only as an instruction to the prompted models, and whether it serves the audiences it names was not tested. Four of its five audiences have no reference data (Chapter 8), and their demonstrations are hand-authored (Appendix A.3).

A human study with readers from these audiences, could show whether the audience parameter helps them specifically. With the help of generative AI models, a corpus of such audience-conditioned examples could be built. A separate model could also be fine-tuned on them, so that trained and instructed audience control could be compared directly. Fine-tuning the prompted model itself, for example parameter-efficiently, is excluded, because it enters RQ1 as an instructed rather than a trained system, and fine-tuning it would remove that distinction.

10 Conclusion

This thesis compared three approaches to automatic text simplification (ATS) at sentence and document granularity: an off-the-shelf simplifier (the *comparison simplifier*), a sequence-to-sequence (seq2seq) model fine-tuned for this thesis, and prompted open-weight large language models (LLMs). All three are served by a browser extension that simplifies web pages in place, and this deployment was audited on twelve authentic pages.

RQ1 asks how the approaches compare. No approach wins outright: the ranking depends on which systems are compared and on which metric is used. Fine-tuning the 139M-parameter base version of Bidirectional and Auto-Regressive Transformers (BART) on a task-matched corpus clearly improves on the untuned checkpoint on System output Against References and against the Input sentence (SARI) and document-level SARI (D-SARI). SARI rises from 21.39 to 37.80 at sentence level (+16.42, confidence interval $[+15.38, +17.50]$, $p < 0.001$), and D-SARI from 14.34 to 35.44 on the complete document test split ($p < 0.001$). These are the only comparisons tested for significance. Against the comparison simplifier, which is three times its size, the fine-tuned checkpoint scores 0.37 SARI lower, well inside the ≈ 2 -point noise floor, so its advantage over that model is serving cost.

Both prompted LLMs score higher than the fine-tuned checkpoint on SARI at sentence level. At document level, the prompted 7B-parameter model scores higher on Learnable Evaluation metric for Text Simplification (LENS) but lower on D-SARI, consistently across three sampling seeds. LENS cannot settle this disagreement, because it is applied outside the sentence-level data it was trained on (Maddela et al., 2023, pp. 16385–16386), so the document-level ordering remains open.

Structured manual inspection found errors that the aggregate scores did not reveal: a document checkpoint trained on less data changed the census year 2010 to 2000 in 27 of 35 affected documents.

RQ2 asks which content-selection and replacement strategies make a deployed simplifier reliable on real pages, and what an audit shows about its robustness. The exploratory site audit gives a mixed answer: replacement is reliable, content selection mostly is, and most emptied links trace back to the model’s edits rather than to the replacement. Replacement preserved the page’s Document Object Model (DOM) on all 24 runs (twelve pages at both granularities): no tag, image source or link target was lost, and reverting

restored the page’s text and attributes every time. Content selection mostly withholds the right text, but its heuristic for code-like text also rejects ordinary prose that contains a semicolon.

The most serious result of the audit is that five sentences were served with a negation deleted, which reversed their meaning, and no guard fired. Prompt injection also succeeded end to end on both prompted LLMs. Whether the prompts’ audience parameter serves the audiences it names was not tested.

This thesis makes four contributions (Section 1.3). It assesses the corpora used and documents data-quality issues in them, such as encoding errors and overlap between training and test data. It compares the three approaches at both granularities in one evaluation harness, including the complete 8,000-document D-Wikipedia test split. It provides a DOM-preserving browser extension in which the approaches can be compared on authentic web pages. Finally, it derives methodological findings about the evaluation of ATS (Section 7.3). These are three observations that plausibly transfer beyond the models used here: the unit a model is trained on should match the unit a deployment feeds it; the choice of benchmark and metric can change a conclusion; and inspecting outputs finds failures that the scores do not show.

I consider the deployment half of this thesis the more useful one. Benchmark scores said nothing about the negation deletions, the withheld prose or the sub-sentence fragments sent to the model; each was found only on closer inspection of the deployed system. In my view, work of that kind is under-reported in ATS research relative to what it costs to find. Since 2010, that research has been dominated by sentence-aligned Simple English Wikipedia data (Xu et al., 2016, p. 402).

The next steps follow from what remains open (Chapter 9): a human study to settle the document-level ordering, a polarity check so that the change guard catches deleted negations, and a language gate before any multilingual training.

The motivating example of Section 1.1 shows why benchmark scores alone are not enough. The sentence-level seq2seq models in this thesis turned the four-word heading “Municipal recycling policy revisions” into “Other websites”. The output may originate in the training corpus, which maps short fragments with little content onto exactly that phrase in 38 pairs. A system pointed at a live page receives inputs whose handling no benchmark score describes.

Finding such failures took something other than a better metric. It took a log of what the model produced and what was served; a harness that runs the extension’s own code rather than a reimplementaion of it; and structured inspection of outputs. The failures and findings that mattered most in this deployment did not show up in the benchmark scores.

References

- Alva-Manchego, F., Martin, L., Bordes, A., Scarton, C., Sagot, B., & Specia, L. (2020). ASSET: A dataset for tuning and evaluation of sentence simplification models with multiple rewriting transformations. In D. Jurafsky, J. Chai, N. Schluter, & J. Tetreault (Eds.), *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 4668–4679). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.acl-main.424>
- Alva-Manchego, F., Martin, L., Scarton, C., & Specia, L. (2019). EASSE: Easier automatic sentence simplification evaluation. In S. Padó & R. Huang (Eds.), *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (emnlp-ijcnlp): System demonstrations* (pp. 49–54). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-3009>
- Alva-Manchego, F., Scarton, C., & Specia, L. (2020). Data-driven sentence simplification: Survey and benchmark. *Computational Linguistics*, 46(1), 135–187. https://doi.org/10.1162/coli_a_00370
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020, July). Language models are few-shot learners [Published in Advances in Neural Information Processing Systems 33 (NeurIPS), 2020]. <https://arxiv.org/abs/2005.14165>
- Cardon, R., & Bibal, A. (2023). On operations in automatic text simplification. In S. Štajner, H. Saggio, M. Shardlow, & F. Alva-Manchego (Eds.), *Proceedings of the second workshop on text simplification, accessibility and readability* (pp. 116–130). INCOMA Ltd., Shoumen, Bulgaria. <https://aclanthology.org/2023.tsar-1.12/>
- Chrome for Developers. (2012). *Content scripts*. Retrieved September 25, 2026, from <https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts>
- Chrome for Developers. (2023, May). *Extension service worker basics*. Retrieved September 26, 2026, from <https://developer.chrome.com/docs/extensions/develop/concepts/service-workers/basics>

- Cohen, E., Bul, I., Inbar, D., & Loewenbach, O. (2026). Simplify-This: A comparative analysis of prompt-based and fine-tuned LLMs. <https://arxiv.org/abs/2601.05794>
- Crossley, S. A., Kyle, K., & McNamara, D. S. (2016). The tool for the automatic analysis of text cohesion (TAACO): Automatic assessment of local, global, and text cohesion. *Behavior Research Methods*, 48(4), 1227–1237. <https://doi.org/10.3758/s13428-015-0651-7>
- FastAPI. (2026, September). *FastAPI Documentation*. Retrieved September 26, 2026, from <https://fastapi.tiangolo.com>
- Google Research. (2026). *Colaboratory: Frequently asked questions*. Retrieved September 26, 2026, from <https://research.google.com/colaboratory/faq.html>
- Google Search Central. (2008, October). *Helping You Break the Language Barrier*. Retrieved September 26, 2026, from <https://developers.google.com/search/blog/2008/10/helping-you-break-language-barrier>
- Hellbusch, J. (2018). Mit barrierefreiem Webdesign zu einer besseren User Experience. In C. Maaß & I. Rink (Eds.), *Handbuch Barrierefreie Kommunikation* (pp. 507–526). Frank & Timme.
- jsdom contributors. (2026). *jsdom*. Retrieved September 25, 2026, from <https://github.com/jsdom/jsdom>
- Jurafsky, D., & Martin, J. H. (2026, August). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3rd) [Online manuscript released August 19, 2026]. <https://web.stanford.edu/~jurafsky/slp3/>
- Kew, T., Chi, A., Vásquez-Rodríguez, L., Agrawal, S., Aumiller, D., Alva-Manchego, F., & Shardlow, M. (2023). BLESS: Benchmarking large language models on sentence simplification. In H. Bouamor, J. Pino, & K. Bali (Eds.), *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 13291–13309). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.emnlp-main.821>
- Kincaid, J. P., Robert P. Fishburne, J., Rogers, R. L., & Chissom, B. S. (1975, February). *Derivation of new readability formulas (automated readability index, fog count and flesch reading ease formula) for navy enlisted personnel* (tech. rep. No. Research Branch Report 8-75). Chief of Naval Technical Training, Naval Air Station Memphis. <https://api.semanticscholar.org/CorpusID:61131325>
- Koehn, P. (2004). Statistical significance tests for machine translation evaluation. In D. Lin & D. Wu (Eds.), *Proceedings of the 2004 conference on empirical methods in natural language processing* (pp. 388–395). Association for Computational Linguistics. <https://aclanthology.org/W04-3250/>

- Laban, P., Vig, J., Kryscinski, W., Joty, S., Xiong, C., & Wu, C.-S. (2023). SWiPE: A dataset for document-level simplification of Wikipedia pages. In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 10674–10695). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.acl-long.596>
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2020). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In D. Jurafsky, J. Chai, N. Schluter, & J. Tetreault (Eds.), *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 7871–7880). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.acl-main.703>
- Maddela, M., Dou, Y., Heineman, D., & Xu, W. (2023). LENS: A learnable evaluation metric for text simplification. In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 16383–16408). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.acl-long.905>
- Martin, L., de la Clergerie, É., Sagot, B., & Bordes, A. (2020). Controllable sentence simplification. In N. Calzolari, F. Béchet, P. Blache, K. Choukri, C. Cieri, T. Declerck, S. Goggi, H. Isahara, B. Maegaard, J. Mariani, H. Mazo, A. Moreno, J. Odiijk, & S. Piperidis (Eds.), *Proceedings of the twelfth language resources and evaluation conference* (pp. 4689–4698). European Language Resources Association. <https://aclanthology.org/2020.lrec-1.577/>
- Martin, L., Fan, A., de la Clergerie, É., Bordes, A., & Sagot, B. (2022). MUSS: Multilingual unsupervised sentence simplification by mining paraphrases. In N. Calzolari, F. Béchet, P. Blache, K. Choukri, C. Cieri, T. Declerck, S. Goggi, H. Isahara, B. Maegaard, J. Mariani, H. Mazo, J. Odiijk, & S. Piperidis (Eds.), *Proceedings of the thirteenth language resources and evaluation conference* (pp. 1651–1664). European Language Resources Association. <https://aclanthology.org/2022.lrec-1.176/>
- Ollama. (2026a). *API introduction*. Retrieved September 26, 2026, from <https://docs.ollama.com/api/introduction>
- Ollama. (2026b). *Importing a model*. Retrieved September 26, 2026, from <https://docs.ollama.com/import>
- Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: A method for automatic evaluation of machine translation. In P. Isabelle, E. Charniak, & D. Lin (Eds.), *Proceedings of the 40th annual meeting of the as-*

- sociation for *computational linguistics* (pp. 311–318). Association for Computational Linguistics. <https://doi.org/10.3115/1073083.1073135>
- Post, M. (2018). A call for clarity in reporting BLEU scores. In O. Bojar, R. Chatterjee, C. Federmann, M. Fishel, Y. Graham, B. Haddow, M. Huck, A. J. Yepes, P. Koehn, C. Monz, M. Negri, A. N  v  ol, M. Neves, M. Post, L. Specia, M. Turchi, & K. Verspoor (Eds.), *Proceedings of the third conference on machine translation: Research papers* (pp. 186–191). Association for Computational Linguistics. <https://doi.org/10.18653/v1/W18-6319>
- Qwen, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., ... Qiu, Z. (2025, January). Qwen2.5 technical report. <https://arxiv.org/abs/2412.15115>
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2023, September). Exploring the limits of transfer learning with a unified text-to-text transformer. <https://arxiv.org/abs/1910.10683>
- Ribeiro, M. T., Wu, T., Guestrin, C., & Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. In D. Jurafsky, J. Chai, N. Schluter, & J. Tetreault (Eds.), *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 4902–4912). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.acl-main.442>
- Rink, I. (2018). Kommunikationsbarrieren. In C. Maa   & I. Rink (Eds.), *Handbuch Barrierefreie Kommunikation* (pp. 29–65). Frank & Timme.
- Romero, P., Li, Z., & Shardlow, M. (2025). Efficient on-device text simplification for Firefox with synthetic data fine-tuning. In M. Shardlow, F. Alva-Manchego, K. North, R. Stodden, H. Saggion, N. Khallaf, & A. Hayakawa (Eds.), *Proceedings of the fourth workshop on text simplification, accessibility and readability (TSAR 2025)* (pp. 105–115). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.tsar-1.7>
- Schubert, K. (2016). Barriereabbau durch optimierte Kommunikationsmittel: Versuch einer Systematisierung. In N. M  lzer (Ed.), *Barrierefreie Kommunikation – Perspektiven aus Theorie und Praxis* (pp. 15–33). Frank & Timme.
- Siddharthan, A. (2014). A survey of research on text simplification. *ITL - International Journal of Applied Linguistics*, 165(2), 259–298. <https://doi.org/10.1075/itl.165.2.06sid>
- Stodden, R. (2026, April). *Automatic German text simplification: Data, evaluation, and models* (Vol. 21). Frank & Timme. <https://doi.org/10.57088/978-3-7329-8700-9>
- Sun, R., Jin, H., & Wan, X. (2021). Document-level text simplification: Dataset, criteria and baseline. In M.-F. Moens, X. Huang, L. Specia,

- & S. W.-t. Yih (Eds.), *Proceedings of the 2021 conference on empirical methods in natural language processing* (pp. 7997–8013). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.emnlp-main.630>
- Tunstall, L., von Werra, L., & Wolf, T. (2023). *Natural Language Processing mit Transformern: Sprachanwendungen mit Hugging Face erstellen* (M. Fraaß, Trans.) [German translation of *Natural Language Processing with Transformers*, Revised Edition]. O’Reilly.
- Vásquez-Rodríguez, L., Nguyen, N. T. H., Przybyła, P., Shardlow, M., & Ananiadou, S. (2024, December). Simple is not enough: Document-level text simplification using readability and coherence. <https://arxiv.org/abs/2412.18655>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023, August). Attention is all you need. <https://arxiv.org/abs/1706.03762>
- W3C. (2026). *Accessible Rich Internet Applications (WAI-ARIA) 1.3* [W3C Working Draft 04 June 2026]. Retrieved September 26, 2026, from <https://www.w3.org/TR/wai-aria-1.3/>
- West, D. M. (2003, September). *State and federal E-government in the United States, 2003* (tech. rep.). Center for Public Policy, Brown University. Providence, RI. <https://www.insidepolitics.org/egovt03us.pdf>
- WHATWG. (2026). *HTML living standard*. Retrieved September 25, 2026, from <https://html.spec.whatwg.org/multipage/>
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., ... Rush, A. (2020). Transformers: State-of-the-art natural language processing. In Q. Liu & D. Schlangen (Eds.), *Proceedings of the 2020 conference on empirical methods in natural language processing: System demonstrations* (pp. 38–45). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
- Xu, W., Callison-Burch, C., & Napoles, C. (2015). Problems in current text simplification research: New data can help (M. Collins & L. Lee, Eds.). *Transactions of the Association for Computational Linguistics*, 3, 283–297. https://doi.org/10.1162/tacl_a_00139
- Xu, W., Napoles, C., Pavlick, E., Chen, Q., & Callison-Burch, C. (2016). Optimizing statistical machine translation for text simplification (L. Lee, M. Johnson, & K. Toutanova, Eds.). *Transactions of the Association for Computational Linguistics*, 4, 401–415. https://doi.org/10.1162/tacl_a_00107
- Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating text generation with BERT. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1904.09675>

- Zhang, X., & Lapata, M. (2017). Sentence simplification with deep reinforcement learning. In M. Palmer, R. Hwa, & S. Riedel (Eds.), *Proceedings of the 2017 conference on empirical methods in natural language processing* (pp. 584–594). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D17-1062>
- Zhu, Z., Bernhard, D., & Gurevych, I. (2010). A monolingual tree-based translation model for sentence simplification. In C.-R. Huang & D. Jurafsky (Eds.), *Proceedings of the 23rd international conference on computational linguistics (Coling 2010)* (pp. 1353–1361). Coling 2010 Organizing Committee. <https://aclanthology.org/C10-1152/>

Appendices

A.1 Link to data and code

The source code, training and evaluation scripts, site-audit artefacts, and model cards are in the following GitHub repository:

<https://github.com/yyvs/ba-simplify>

The four site-audit artefacts behind Tables A.4 to A.7 share one directory and one run stamp,

`research/results/site_audit/site_audit_20260826T044008Z`

and differ only in their suffix:

- `.json`, the twenty-four runs and their per-page totals;
- `_items.jsonl`, one row per registered leaf, 11,867 of them;
- `_units.jsonl`, one row per request, 2,571 of them;
- `_labelling.jsonl`, the 130 hand-labelled items of Appendix A.6.

The prompt templates of Appendix A.2 are `backend/prompting.py`, the demonstration sampler is `research/select_fewshot.py`, and the audit harness is `research/audit_sites.mjs`.

A.2 Prompt templates

A.2.1 Sentence granularity

The instruction below is BLESS Prompt 2 (Kew et al., 2023, p. 13294) reproduced verbatim, with the single parameterisation described in Section 4.4: the paper’s hard-coded noun phrase *non-native speakers of English* becomes the slot `{target_audience}`. Rendering the slot with its default value therefore returns the published prompt character for character, which a unit test asserts.

Please rewrite the following complex sentence in order to make it easier to understand by {target_audience}. You can do so by replacing complex words with simpler synonyms (i.e. paraphrasing), deleting unimportant information (i.e. compression), and/or splitting a long complex sentence into several simpler ones. The final simplified sentence needs to be grammatical, fluent, and retain the main ideas of its original counterpart without altering its meaning.

The slot accepts only a closed set of values rather than free text, so that evaluation runs stay reproducible, and the slot cannot carry injected instructions alongside the page content. The set has five values:

```
non-native speakers of English (default)
children aged 8 to 12
adult readers with low literacy skills
readers with cognitive disabilities
a general adult audience with no specialist knowledge
```

The rendered prompt places the instruction first, then the three demonstrations of Appendix A.3 as **Complex:**/**Simple:** pairs, then the input and a dangling **Simple:** prefix for the model to continue from, with a blank line between blocks:

```
<instruction>

Complex: <demonstration 1 source>
Simple: <demonstration 1 reference>

Complex: <demonstration 2 source>
Simple: <demonstration 2 reference>

Complex: <demonstration 3 source>
Simple: <demonstration 3 reference>

Complex: <input sentence>
Simple:
```

A.2.2 Document granularity

The document template has no published source. It adapts the sentence template in the three ways recorded in Section 4.4, with the frame **Text:**/**Simplified text:** in place of **Complex:**/**Simple:**, and carries no demonstrations by default.

Please rewrite the following text in order to make it easier to understand by {target_audience}. You can do so by replacing complex words with simpler synonyms (i.e. paraphrasing), deleting unimportant information (i.e. compression), splitting long complex sentences into several simpler ones, and merging or reordering sentences where that makes the whole text clearer. The result needs to be grammatical, fluent, and retain the main ideas of the original without altering its meaning. Return only the rewritten text as continuous prose -- no commentary, no headings, no bullet points, and no introduction such as "Here is the simplified text".

A.2.3 Decoding and stop sequences

Table A.1 gives the two decoding configurations. Serving is greedy so that re-simplifying a page produces the same text, while evaluation reproduces BLESS’s own reported sampling settings (Kew et al., 2023, p. 13293) so that the numbers are comparable to the paper.

Table A.1
Decoding Configurations for Serving and for Evaluation

Configuration	Temperature	Top- p	Token cap	Stop sequences
Serving, in the extension	0.0	n/a	200	<code>\nComplex:</code> , <code>\nSimple:</code> , <code>\n\n</code>
Evaluation, per BLESS	1.0	0.9	100	<code>\nComplex:</code> , <code>\nSimple:</code> , <code>\n\n</code>
Either, at document granularity	unchanged			<code>\nText:</code> , <code>\nSimplified text:</code>

Note. The document row overrides only the stop sequences, since the sentence list’s blank-line stop would cut every multi-paragraph output at its first paragraph. The three evaluation seeds are supplied per run, not fixed in the template.

A.3 In-context demonstrations

Table A.2 gives the three demonstrations used for the default audience. They are sampled from the ASSET validation split at seed 42 under the four selection rules of Section 4.4. Table A.3 gives the hand-authored demonstrations of the four other audiences.

Table A.2*In-Context Demonstrations and the Operations They Cover*

Index	Complex (source)	Simple (reference)	Operation
285	To avoid adversaries from guessing the key using a brute-force attack, the key space is usually designed to be extremely large.	To keep foes from guessing the key using a brute-force attack, the key space is designed to be extremely large.	paraphrase
1516	She is the daughter of well-known Hong Kong singer Teresa Carpio, with whom she has performed on stage as a backup singer.	She is the daughter of famous Hong Kong singer Teresa Carpio, with whom she performed as a backup singer.	compression
1116	Other Mario sports games include the Camelot-developed series Mario Golf and Mario Tennis, and, respectively, the baseball and soccer games Mario Superstar Baseball and Super Mario Strikers.	Other Mario sports games are the Mario Golf and Mario Tennis. Also, the baseball and soccer games Mario Superstar Baseball and Super Mario Strikers.	split

Note. Row 285 also drops the hedge *usually*, which is a meaning change the reference makes and the instruction does not license. The four non-default audiences have no parallel corpus, so their pools are hand-authored (Table A.3), rest on a weaker footing, and are not comparable to BLESS.

A.4 The twelve audited pages

Table A.4 identifies the pages behind Section 6.6. They were chosen by category before any was run, two per category, and picked informally within each category, without a sampling procedure. Each page was captured once and both runs read the same cached copy, so the twenty-four runs are comparable and reproducible after the live pages change.

A.5 Per-page coverage counts

Figure 6.6 plots skip shares; Table A.5 gives the counts underneath them. The same 95 % skip share means far more withheld leaves on a page registering 1,749 than on one registering 39, a difference the aggregate totals of Section 6.6 cannot show.

Table A.3*Hand-Authored Demonstrations for the Four Non-Default Audiences*

Audience	Simple (hand-authored)
Source 1: The commission subsequently determined that the allegations were unsubstantiated and dismissed the complaint in its entirety.	
children aged 8 to 12	The group looked at the claims. They found no proof, so they said no to the complaint.
adult readers with low literacy skills	The commission checked the claims. They found no proof. They rejected the complaint.
readers with cognitive disabilities	A group of people checked the complaint. They found no proof. They said no to it.
a general adult audience with no specialist knowledge	The commission found no evidence for the claims and threw out the complaint.
Source 2: Owing to inclement weather conditions, the outdoor ceremony was relocated to an adjacent indoor venue.	
children aged 8 to 12	The weather was bad. So the party moved inside to a room next door.
adult readers with low literacy skills	The weather was bad. The ceremony moved indoors. It took place in a room next door.
readers with cognitive disabilities	The weather was bad. The event went inside. It was in the room next door.
a general adult audience with no specialist knowledge	Bad weather forced the outdoor ceremony to move to an indoor venue next door.
Source 3: He is widely regarded as one of the most influential composers of the Romantic era, a period spanning roughly from 1800 to 1910.	
children aged 8 to 12	He wrote music a long time ago. Lots of people think he was one of the best music writers of his time.
adult readers with low literacy skills	He wrote music. Many people say he was one of the best. He lived in a time called the Romantic era.
readers with cognitive disabilities	He made music. People liked his music very much. He lived long ago.
a general adult audience with no specialist knowledge	He is seen as one of the most influential composers of the Romantic era, which ran from about 1800 to 1910.

Note. The texts are those in `backend/prompting.py`.

Table A.4*The Twelve Audited Pages, by Category and URL*

Category	Key	Page and URL as retrieved
news	<code>bbc</code>	BBC News article bbc.com/news/articles/c0k3700zljo
news	<code>tagesschau</code>	tagesschau news article (German) tagesschau.de/ausland/asien/gaza-wiederaufbau-100.html
encyclopedic	<code>wikipedia</code>	Wikipedia: Photosynthesis, as served by the article HTML API en.wikipedia.org/api/rest_v1/page/html/Photosynthesis
encyclopedic	<code>sep</code>	Stanford Encyclopedia of Philosophy: Consciousness plato.stanford.edu/entries/consciousness/
government	<code>govuk</code>	GOV.UK guidance: Living in Germany gov.uk/guidance/living-in-germany
government	<code>europa</code>	Your Europe: entry and exit rules for EU citizens europa.eu/youreurope/citizens/travel/entry-exit/eu-citizen/index_en.htm
e-commerce	<code>ikea</code>	IKEA product page: BILLY bookcase ikea.com/gb/en/p/billy-bookcase-white-00263850/
e-commerce	<code>allbirds</code>	Allbirds storefront, redirected from a product page allbirds.com/collections/mens
documenta- tion	<code>mdn</code>	MDN reference for the CSS <code>flex</code> shorthand property developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/flex
documenta- tion	<code>pydocs</code>	Python documentation: coroutines and asyncio tasks docs.python.org/3/library/asyncio-task.html
single-page app	<code>reactdev</code>	react.dev tutorial: Thinking in React react.dev/learn/thinking-in-react
single-page app	<code>nextjs</code>	Next.js documentation: Installation nextjs.org/docs/app/getting-started/installation

Note. All twelve were fetched on 26 August 2026 between 03:34 and 03:39 UTC, cached with the SHA-256 of their HTML truncated to 16 hexadecimal digits, and audited against those copies at commit `5989ab4`. URLs are shown without their scheme and `www` prefix; the links carry the full address. Two URLs resolved elsewhere than requested: MDN served the same `flex` page under a `Reference/Properties` path, and Allbirds redirected the requested product page to a collection listing. Most large shops answered the scripted fetch with a bot challenge, so the two audited shops need not be typical of e-commerce sites. The two single-page apps were audited as the HTML the server delivers, since `jsdom` runs no page JavaScript by default (jsdom contributors, 2026).

Table A.5*Per-Page Coverage Counts in Both Cuts*

Category (page)	Sentence cut				Document cut			
	Found	Sent	Changed	Skip%	Found	Sent	Changed	Skip%
news (<code>bbc</code>)	384	85	64	77.9	42	11	9	73.8
news (<code>tagesschau</code>)	290	65	42	77.6	41	6	3	85.4
encyclopedic (<code>wikipedia</code>)	1,220	278	249	77.2	995	46	45	95.4
encyclopedic (<code>sep</code>)	612	480	405	21.6	214	99	93	53.7
government (<code>govuk</code>)	463	311	206	32.8	172	58	39	66.3
government (<code>europa</code>)	562	246	139	56.2	39	12	8	69.2
e-commerce (<code>ikea</code>)	416	53	34	87.3	275	10	4	96.4
e-commerce (<code>allbirds</code>)	845	213	172	74.8	330	3	3	99.1
documentation (<code>mdn</code>)	1,749	85	44	95.1	226	11	6	95.1
documentation (<code>pydocs</code>)	956	230	148	75.9	754	29	23	96.2
single-page app (<code>reactdev</code>)	354	82	59	76.8	130	11	10	91.5
single-page app (<code>nextjs</code>)	926	96	60	89.6	194	15	15	92.3
Total	8,777	2,224	1,622	74.7	3,412	311	258	90.9

Note. *Found* is registered candidates, *Sent* those that passed the filters and reached the backend, and *Changed* those the model altered. *Skip%* is *Found* minus *Sent* over *Found*; in the total row it is computed from the summed counts, so it is weighted by page size. Within the body, bold marks the extreme skip share at each end of each cut’s range. The two cuts’ skip shares are not comparable with each other (Section 6.6). The two *Found* columns sum to 12,189 against 11,867 rows in the item artefact, because a leaf registered in both cuts is one row there and two registrations here.

A.6 Hand-labelling codebook

The 130 items behind Figure 6.7 were sampled and labelled under the scheme below. The sample is stratified by page rather than drawn from the pool, three items per page per cut, because the question is how the filters behave across page types. A flat draw from the skipped pool would be mostly Wikipedia and MDN. Each item’s rank is the SHA-256 of its own text, so the sample is deterministic without a random seed and each label stays attached to its item.

The skipped side is restricted to items the filters actually decided about, meaning prose-length text of at least eight words inside the page’s own content area. Section headings are excluded because the document cut skips those by design. A sample of skipped navigation labels would be labelled correct almost everywhere and measure nothing. The unchanged side draws from every unit the model was given and handed back unaltered.

Table A.6

Hand-Labelling Codebook: Verdict Criteria and Counts

Verdict	Criterion applied	<i>n</i>
<i>Skipped items (n = 69):</i> text the filters withheld from the model		
correct	Withholding was right: the text is not prose, or is boilerplate, or is actual code	48
should-have-been-sent	Running prose with no code in it, withheld anyway	15
correct-but-costly	Prose inside a subtree the page itself opted out of translation; the extension honours the opt-out correctly and the reader loses the content regardless	6
<i>Unchanged items (n = 61):</i> text the model received and left unchanged		
already-simple	The text was already plain, so leaving it unaltered is the correct behaviour	49
should-have-been-simplified	Complex prose left essentially unchanged	9
language-unsupported	German text sent to an English-only checkpoint, with no language gate to stop it	3

Note. Both sides carried an `unclear` option, which no item received.

`correct-but-costly` and `language-unsupported` were added during labelling, for items that were neither correct nor a defect of the filter. All six `correct-but-costly` items are on one page. One annotator, the author, labelled all 130 items, so there is no agreement statistic (Chapter 8).

A.7 Unguarded meaning inversions

Table A.7 gives the five cases Section 6.6 reports: output that repeats the input with a negation deleted. No guard fired on any of them.

Table A.7
Unguarded Meaning Inversions in the Sentence Cut

Page	Input	Output
<code>allbirds</code>	Final Sale items, or returns that do not comply with our terms and conditions cannot be returned or exchanged.	Final Sale items or returns that do not comply with our terms and conditions can be returned or exchanged.
<code>allbirds</code>	If you decide not to buy return coverage you can still return your items, but you will be responsible for the cost of the return shipping label.	If you decide to buy return coverage you can still return your items, but you will be responsible for the cost of the return shipping label.
<code>pydocs</code>	The aws iterable must not be empty.	The aws iterable must be empty.
<code>tagesschau</code>	Ein koordinierter Wiederaufbau ist nicht in Sicht.	Ein koordinierter Wiederaufbau ist in Sicht.
<code>tagesschau</code>	Der Weg hin zu den eigenen vier Wänden war nicht einfach.	Der Weg hin zu den eigenen vier Wänden war einfach.

Note. Bold marks the deleted negation (emphasis added). Each sentence is quoted from the unit that contained it; in two cases the rest of the unit was also edited. Detection required the output sentence to equal the input minus exactly one negation token, so five is a lower bound (Section 6.6). The German rows read “A coordinated reconstruction is not in sight” and “The road to a home of one’s own was not easy”.

A.8 Recorded outputs of the input-convention comparison

Table A.8 gives the outputs behind Table 5.2, in the document model’s lower-cased, pre-tokenised output convention. The input was one hand-constructed three-fact document, which was not kept.

Table A.8*Recorded Outputs of the Document Model Under Five Input Conventions*

Input convention	Recorded output	Assessment
corpus convention (lowercased, pre-tokenised, no newlines)	achtkarspelen is a municipality in friesland in the northern netherlands . it has a population of about 28,000 people .	faithful; two of the three facts kept
ordinary mixed-case prose	achtkarspelen is a municipality in the province of friesland in the northern hemisphere .	factual hallucination (<i>Netherlands</i> became <i>hemisphere</i>)
corpus convention with <code>\n</code> separators	achtkarspelen is a municipality in the northern part of the country of friesland .	two facts lost, and Friesland, a province, called a country
ordinary prose normalised to the corpus convention	identical to the first row	the convention, not the content, changes the output
heading text included in the section body	begins with <code>history</code> .; only the opening was recorded	the heading echoed into the body

Note. $n = 1$, one checkpoint, one set of decoding settings; judged by inspection. Source: the project’s working notes at the time of the comparison.

Statement of Originality

I hereby confirm that I have composed the accompanying thesis by myself, without contributions from any sources other than those cited in the text and the below declaration. This applies also to all tables and code included in the thesis.

Declaration of AI Usage

I, Yunus Oscar Renz (Matrikelnummer 2952305), hereby confirm that I have read and acknowledged the AI policy of the Institute for Linguistics, HHU (June 2025). **I hereby declare that for my Computational Linguistics Bachelor's Thesis**, with the title *From Benchmark to Browser: A Comparative Evaluation of Automatic Text Simplification Approaches for DOM-Preserving Web Deployment*, with apl. Prof. Dr. Wiebke Petersen as examiner, **I have used AI**.

The following list contains all tools that I have used as well as a short description of how and for what purpose these tools were used.

Tool	Ver- sion/Date	Purpose
Gemini Notebook	free version, Sep 2026	Literature review, source summarization, comparison of research papers
ChatGPT	free version, Sep 2026	Conceptual explanations, data analysis, academic writing support
Claude	test version, Sep 2026	Thesis structuring, critical review, reasoning, academic writing support
Claude Code	test version, Sep 2026	Programming assistance, code development, debugging, LaTeX support
GitHub Copilot	free version, Sep 2026	Code completion, implementation assistance, debugging, refactoring

Düsseldorf, September 30th, 2026

Yunus Oscar Renz